TELEPÍTÉSI ÚTMUTATÓ

OpenHDID – Ügyfélszolgálati hívóazonosítás

Az OpenHDID konténeres telepítése: követelmények, telepítés a telepítővel vagy lépésről lépésre, a beállítások működése a kész csomagban, hálózat és HTTPS, adatbázis, ütemezett feladatok, naplók, a külső rendszerek bekötése és az átvételi ellenőrzés.

VERZIÓ
1.15.3 · 2026. november 9.

KIADÁS
OpenHDID 1.15.3 · ghcr.io/adamfhun/openhdid

CÉLKÖZÖNSÉG
rendszerüzemeltető, rendszergazda

KAPCSOLÓDÓ
üzemeltetési kézikönyv, felhasználói kézikönyv

TARTALOM

1 Áttekintés
2 Követelmények
3 Telepítés lépésről lépésre
4 A beállítások működése
5 Környezeti változók
6 Hálózat, kiszolgálónév, HTTPS
7 Adatbázis (MariaDB)
8 Háttérfolyamatok és ütemezés
9 Naplók
10 Első beállítás a felületen
11 Külső rendszerek bekötése
12 Ellenőrzés és biztonsági átvétel
13 Kipróbálás demóadatokkal
14 Frissítés új verzióra
15 Használati profilok, magas rendelkezésre állás, Kubernetes
16 A telepítő (install.sh)
R Rövidítésjegyzék

1 Áttekintés

Az OpenHDID egyetlen konténer-image-ként érkezik (`ghcr.io/adamfhun/openhdid` , `linux/amd64` és `linux/arm64`). Ugyanaz az image minden szerepet ellát, a Compose-fájl a parancs szerint indítja a szolgáltatásokat:

SZOLGÁLTATÁS	PARANCS	FELADAT
`web`	`web`	nginx (mainline, headers-more, TLS 1.3) és PHP-FPM egy konténerben: ügyfélportál, munkatársi panel, gépi API-k. Az nginx és a PHP-FPM Unix-socketen beszél, FastCGI-port nem nyílik.
`worker`	`worker`	Üzenetsor-feldolgozó: e-mail, SMS, életjel. Sorlista nélkül előbb az elsőbbségi sort (`HDID_PRIORITY_QUEUE` , alapból `priority` : a rövid életű belépési kódok), utána a `default` sort viszi.
`sync-worker`	`worker sync`	A panelről indított ügyféltörzs-átvétel külön soron (legfeljebb 20 perc), hogy ne tartsa fel az üzeneteket.
`scheduler`	`scheduler`	Az ütemező a cron helyett (`php artisan schedule:work`). Pontosan egy példány fusson.
`migrate`	`migrate`	Minden indításkor egyszer lefut: megvárja az adatbázist, lefuttatja a migrációkat és a szerepkör-seedert (amely csak bővít: a szerepkörök kézi módosításait nem állítja vissza), majd kilép. A többi szolgáltatás csak a sikeres lefutása után indul.
`mariadb`	–	Beépített MariaDB 12.3 LTS a `db` profillal. Külső adatbázisnál elhagyható.
`valkey`	–	Valkey (Redis-kompatibilis, BSD-licencű) a `redis` profillal: gyorsítótár, üzenetsor, zárak és korlátszámlálók; memóriakorláttal, betelt tárnál kulcs eldobása nélkül (`maxmemory-policy noeviction`).
`valkey-sessions`	–	A munkamenetek saját Valkey-je a `redis` profillal (`maxmemory-policy allkeys-lru` , ötpercenkénti mentés): betelt tárnál a legrégebben használt munkameneteket szorítja ki, így egy belépőoldal-áradat a sort, a zárakat és a korlátszámlálókat nem állítja meg.

A telepítési mappa tartalma: `compose.yaml` ; `.env` (alkalmazás); `db.env` (adatbázis-rendszergazda és migrációs fiók); `tls/` (saját tanúsítvány és kulcs); választhatóan `ca/` (belső tanúsítványkiadók). Az image nem tartalmaz beállítást, kulcsot, tanúsítványt és dokumentációt. A dokumentáció a forrásrepó `docs` mappájában és minden kiadás mellékleteként érhető el.

Az image tartalma:

- Alpine Linux 3.24, OpenSSL 3.5 (LTS);

- PHP 8.5 FPM csak a szükséges kiterjesztésekkel: pdo_mysql, intl a teljes ICU-adattal, bcmath, zip, soap, pcntl, redis, OPcache;
- forrásból fordított nginx mainline, csak a használt modulokkal és a beépített headers-more modullal;
- az alkalmazás az éles függőségeivel.

Ami kimaradt: Composer, Node.js, git, fordítóeszközök, PHP-forrás, PEAR, curl-parancs, dokumentáció és tesztek. A konténer a `www-data` felhasználóval (uid 82) fut, csak olvasható gyökér-fájlrendszerrel; írni csak a `storage` kötetre és a memóriában lévő `/tmp`-re tud. Minden szolgáltatásnak memóriaplafonja van (`mem_limit`), és a plafonok összege belefér a használati profil gépébe (15. fejezet).

---

2 Követelmények

ÖSSZETEVŐ	KÖVETELMÉNY	MEGJEGYZÉS
Kiszolgáló	Biztonsági támogatást kapó Linux, x86_64 vagy arm64	rendszeres OS-frissítéssel, szinkronizált órával (NTP): a HMAC-aláírás 300 mp-es időablaka és az SSO-tokenek ellenőrzése is erre épül
Docker	Docker Engine 27+ és Docker Compose 2.24+	a Compose-fájl a beépített `configs.content` és a `depends_on.required` elemet használja; rootless Docker is megfelelő (a 3. fejezet „Rootless Docker” része szerint). **Podman nem támogatott:** a python `podman-compose` a profilhoz kötött `depends_on` miatt `KeyError: 'mariadb'` hibával áll meg, a docker-compose szolgáltató a Podman socketjén nem lett végigvive. Podmanos gépen a Podman eltávolítása és a Docker CE telepítése az út (lent); a közben beállított `DOCKER_HOST` változót és docker contextet is el kell távolítani
Erőforrás	a használati profil szerint, a gépméreteket a 15. fejezet táblázata adja: kipróbáláshoz egy gép 2 vCPU, 8 GB RAM, 20 GB lemez; **5k** egy gép 4 vCPU, 10 GB RAM, 80 GB lemez; **10k egy gépen** 8 vCPU, 16 GB RAM, 120 GB lemez; **10k** két alkalmazásgép (4 vCPU, 6 GB) külső MariaDB-vel és két külső Valkey-vel, vagy helyettük egy infra gép (4 vCPU, 10 GB, 100 GB); **20k** Kubernetes	a méretek a normál csúcsterhelés kétszeresére szólnak (15. fejezet); a térfogati DDoS elleni védelem a szolgáltató vagy egy előtét feladata
Név és tanúsítvány	DNS-név, hozzá tanúsítvány teljes lánccal és kulccsal (PEM)	ECDSA P-256 vagy legalább 2048 bites RSA; jelszóval védett kulcs nem használható
Adatbázis	beépített MariaDB 12.3 LTS vagy külső MariaDB/MySQL	utf8mb4; külső kiszolgálóhoz TLS-kapcsolat ajánlott

Docker telepítése. A disztribúció saját Docker-csomagja gyakran régi vagy hiányzik (AlmaLinux, RHEL), ezért a Docker saját tárolója ajánlott. Példa AlmaLinux 9-en (AlmaLinux 10-en a tároló felvétele `dnf config-manager addrepo --from-repofile=…` alakú):

```
sudo dnf remove -y podman podman-docker podman-compose buildah   # ha fent vannak
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker <üzemeltető felhasználó>
```

A `docker` csoport tagsága csak új bejelentkezéssel él (az `id` parancs mutatja), és root-joggal ér fel: ez a felhasználó csak a telepítés üzemeltetésére szolgáljon. Ellenőrzés: a `docker version` és a `docker compose version` kliens- és szerveroldalt is mutat.

Hálózat.

- Bejövő: 443 a böngészőknek, a telefonközpontnak és a mobil kiszolgálónak; 80 csak a böngészős HTTPS-átirányításhoz kell.
- Kimenő, a konténerekből: Exchange (EWS) vagy SMTP, Ozeki SMS-átjáró, Microsoft Entra és ADFS, az ügyféltörzs gépi felülete, valamint a DNS.
- Az image letöltéséhez a `ghcr.io` kell; zárt hálózatban saját registrybe tükrözhető.
- **Proxy mögött:** a Docker-démon nem örökli a felhasználó `http_proxy` / `https_proxy` változóit, a letöltés 30 másodperc után időtúllépéssel elhal; a proxyt systemd drop-innal kell megadni (üzemeltetési kézikönyv 6. fejezet). Az alkalmazás kimenő kapcsolatai (Exchange, SMS-átjáró, ADFS, Entra, ügyféltörzs) ettől függetlenül a `HDID_HTTP_PROXY` kulccsal mennek proxyn át, a `HDID_NO_PROXY` listában felsorolt belső kiszolgálók kivételével (lásd a „Külső rendszerek” táblázatot). A konténer örökölt `HTTP_PROXY` / `HTTPS_PROXY` változóit az alkalmazás nem használja.

> **Kliens-IP a Docker mögött.**
> Az IP-alapú forgalomkorlát és az auditnapló a kérés valódi forráscímére épül. IPv4-en a Docker megőrzi a kliens címét. IPv6-on, kikapcsolt `ip6tables` vagy a userland-proxy használata mellett viszont a konténer a Docker-átjáró címét látja. Telepítés után a `docker compose logs web` access logjában ellenőrizze, hogy az `ip` mező a valódi kliens címét mutatja-e. Ha nem, kapcsolja be a Docker IPv6-NAT-ját, vagy tegye a webet egy terheléselosztó mögé (`OPENHDID_TLS=off` , 6.5 pont). Ez a kéréskorlátnál is döntő: az nginx kliens-címenként korlátoz (6.8 pont), így ha minden kérés ugyanarról a címről érkezik, a korlát az egész forgalomra hat.

---

3 Telepítés lépésről lépésre

Telepítés a telepítővel (ajánlott)

Az `install.sh` ennek a fejezetnek minden lépését elvégzi, és minden profilra ugyanaz a belépési pont (kipróbálás, 5k egy gépen, 10k egy gépen, 10k két alkalmazásgéppel és infra géppel, 20k Kubernetes): megkérdezi, mit szeretne telepíteni, a kiadás fájljait ellenőrzőösszeggel tölti le, a titkokat maga készíti, a `.env`-et és a `db.env`-et 600-as joggal írja, a tanúsítványt a konténer felhasználójával (rootless

Dockerrel is, `sudo` nélkül) teszi a helyére, elindítja a vermet és ellenőrzi; minden lépésnél kimondja, mit és miért csinál, és a gépen kívüli teendőket (DNS, tűzfal, terheléselosztó, NFS, replika) pontos parancsokkal írja ki. Csomagot nem telepít, rootot nem kér, meglévő telepítésre nem telepít.

```
sudo mkdir -p /opt/openhdid && sudo chown "$(id -un)" /opt/openhdid && cd /opt/openhdid
base=https://github.com/adamfhun/openhdid/releases/latest/download
curl -fsSL -O "$base/install.sh" -O "$base/SHA256SUMS"
sha256sum -c --ignore-missing SHA256SUMS      # install.sh: OK
chmod 755 install.sh
./install.sh
```

A kérdéseket szerepenként, a kétgépes 10k sorrendjét (infra, A, B) a válaszfájllal, a Kubernetes-kimenetet, az ellenőrzést (`./install.sh check`) és a javítást (`./install.sh fix`) a 16. fejezet írja le. A kézi út, lent, ugyanide vezet; a telepítő után is érvényes leírás arról, mi hol van.

Kézi telepítés

A példák a `/opt/openhdid` mappát használják. A parancsokat a `docker` csoport tagja futtatja (2. fejezet); az alkalmazás maga nem root felhasználóként fut.

1. **A telepítési fájlok letöltése** a legfrissebb kiadásból. A kiadások böngészhető oldala a `https://github.com/adamfhun/openhdid/releases/latest`; a lenti alapcím mindig a legfrissebb kiadás mellékleteire mutat, egy adott verzió a `https://github.com/adamfhun/openhdid/releases/download/v<verzió>` alapcímmel választható. Az alapcím önmagában nem megnyitható oldal (404-et ad), csak a fájlnévvel együtt él. A mappa tulajdonosa a parancsokat futtató fiók (`id -un`; a `$USER` változót nem minden shell állítja be). A `.env` és a `db.env` titkokat tartalmaz, ezért csak a tulajdonos olvashatja:

   ```
   sudo mkdir -p /opt/openhdid && sudo chown "$(id -un)" /opt/openhdid && cd /opt/openhdid
   base=https://github.com/adamfhun/openhdid/releases/latest/download
   curl -fsSL -O "$base/compose.yaml"
   curl -fsSL -o .env "$base/env.example"
   curl -fsSL -o db.env "$base/db.env.example"
   curl -fsSL -O "$base/backup.sh"
   curl -fsSL --create-dirs -o profiles/5k/compose.yaml "$base/profile-5k.yaml"
   chmod 600 .env db.env
   chmod 755 backup.sh
   ```

   A `backup.sh` a mentés és a visszaállítás szkriptje (üzemeltetési kézikönyv 7. fejezet); a `profiles/5k/compose.yaml` a használati profil kiegészítő fájlja, amelyre a letöltött `.env` `COMPOSE_FILE` sora mutat (15. fejezet; a 10k profilé a `profile-10k.yaml` melléklet). Enélkül a Compose minden parancsa a `compose file "profiles/5k/compose.yaml" set by COMPOSE_FILE environment variable is invalid` hibával áll meg (`TROUBLESHOOTING.md` 15. fejezet). A profilfájl a kiadás része, frissítésnél a `compose.yaml`-lal együtt cserélődik (14. fejezet).

2. **Kötelező értékek a `.env` fájlban:**

   - `OPENHDID_VERSION`: a letöltött `env.example`-ben már a kiadás pontos verziószáma áll (a `releases/latest` mindig a legfrissebb kiadás fájljait adja), ezt nem kell kikeresni; `latest`-re ne írja át;
   - `APP_URL`: a webhely HTTPS-címe, ebből képződik a kiszolgálónév is;

    - `DB_PASSWORD`: a futási adatbázis-fiók jelszava;
    - `REDIS_PASSWORD`: a Valkey jelszava (a munkamenet, a gyorsítótár és az üzenetsor alapból Valkey-n fut, a beépített `valkey` és `valkey-sessions` is ezzel a jelszóval védett; a 4. lépés parancsa ehhez is ad értéket);
    - `COMPOSE_FILE`: a használati profil kiegészítő fájlja, alapból `compose.yaml:profiles/5k/compose.yaml` (15. fejezet); a profil adja a PHP-FPM, a feldolgozó- és az adatbázis-méreteket;
    - a levelezés, az SMS és az ügyféltörzs beállításai (5. fejezet).

   A `db.env` fájlban a `MARIADB_ROOT_PASSWORD` és a migrációs fiók jelszava (`DB_PASSWORD`) kell. Mindhárom jelszó egyedi, hosszú és véletlenszerű legyen, például `openssl rand -base64 32`. Mindhárom titok fájlból is megadható (`MARIADB_ROOT_PASSWORD_FILE`, `DB_PASSWORD_FILE` a `db.env`-ben és a `.env`-ben), lásd a 4. fejezet „Titkok fájlból” pontját.

3. **Mappák és az image letöltése.** A tanúsítvány (`tls`) és a belső tanúsítványkiadók (`ca`) mappáját a telepítő fiók hozza létre: a Compose a hiányzó csatolt mappát root tulajdonnal hozná létre, és a tanúsítvány másolása a következő lépésben elbukna. Az image a `ghcr.io`-ról jön (zárt hálózatban a 2. fejezet szerint tükrözött registryből); az alkalmazáskulcsot is ez az image készíti, ezért a letöltés már itt kell:

   ```
   mkdir -p tls ca
   docker compose pull
   ```

4. **Alkalmazáskulcs és Valkey-jelszó.** Egyszer kell létrehozni. A parancs két sort ír ki (`APP_KEY=…` és `REDIS_PASSWORD=…`); mindkettőt a `.env` azonos nevű sorába kell írni, az alkalmazáskulcsot a mentésektől elkülönített, védett helyen is meg kell őrizni:

   ```
   docker compose run --rm --no-deps web key
   ```

   Image nélkül ugyanilyen értékeket ad: `echo "base64:$(openssl rand -base64 32)"` és `openssl rand -hex 24`. A kulccsal titkosított adatok (biztonsági válaszok, kódmásolatok, API-tokenek) nélküle nem olvashatók vissza. Meglévő telepítésen soha ne cserélje.

5. **Tanúsítvány.** A teljes lánc és a kulcs a `tls` mappába kerül. A konténer felhasználójának (uid 82) olvashatónak kell lennie:

   ```
   cp /utvonal/fullchain.pem /utvonal/privkey.pem tls/
   sudo chown 82:82 tls/fullchain.pem tls/privkey.pem
   sudo chmod 444 tls/fullchain.pem && sudo chmod 400 tls/privkey.pem
   ```

   Rootless Dockerrel a `sudo chown 82:82` nem jó: ott a konténer 82-es felhasználója a gazdagépen egy altartományi azonosító (például `165617`), a tulajdonost a konténer névterén belül kell beállítani; a parancsot és a számítást a fejezet végén a „Rootless Docker” rész adja. Sudo nélkül, rootful Dockeren ugyanaz a parancs használható.

6. **Belső tanúsítványkiadók** (választható). Ha az Exchange, az ADFS, az ügyféltörzs vagy a külső adatbázis belső CA-val hitelesít, a CA-tanúsítványt tegye a `ca` mappába (`*.crt` vagy `*.pem`). Indításkor a rendszer tanúsítványtára mellé kerül; a TLS-ellenőrzés kikapcsolására nincs szükség.

7. **Indítás** (az image a 3. lépésben már letöltődött):

```
docker compose up -d
docker compose ps
```

Az első indításkor sorrendben:

   1. a MariaDB létrehozza az adatbázist és a két fiókot;
   2. a `migrate` lefut, majd kilép (`exited (0)`);
   3. a `web` , `worker` , `sync-worker` és `scheduler` elindul; a `web` állapota egy percen belül `healthy` .

8. **Az első rendszergazda** (jelszót kér, legalább 12 karakter):

```
docker compose exec web php artisan hdid:make-admin admin@example.org --
name="Rendszergazda"
```

Éles környezetben a parancs `.test` tartományú címet nem fogad el, és figyelmeztet, ha az adatbázisban demóadat van (13. fejezet). A parancs helyi ügyféltörzsbeli tételt is létrehozza, így a belépési szabály teljesül; a későbbi ügyféltörzs-átvételek ezt a helyi tételt nem tekintik hiányzónak, és ha az e-mail-cím az ügyféltörzsben is megjelenik, a fiókot átkötik rá. A jelszót rejtett prompt kéri; ha a belépés „hibás felhasználónév vagy jelszó” hibát ad, a parancs ugyanarra az e-mailre újrafuttatva új jelszót állít, a valódi elutasítási ok pedig az auditnaplóból kérdezhető le (üzemeltetési kézikönyv 6. fejezet). Ha a `web` még nem fut, a parancs `docker compose run --rm --no-deps web artisan hdid:make-admin …` alakban is megy.

9. **Első ellenőrzés és a telefonközpont kulcsa** (a kulcs egyszer jelenik meg):

```
docker compose exec web php artisan hdid:health
docker compose exec web php artisan hdid:api-key:create "Prémium telefonközpont"
callcenter
```

10. Belépés a `https://<kiszolgáló>/admin` címen, majd az első beállítás a 10. fejezet szerint, végül az átvételi ellenőrzés a 12. fejezet szerint.

Rootless Docker

A rendszer rootless Dockerrel is fut: a démont nem a root, hanem az üzemeltető felhasználó futtatja, a konténerek felhasználói névtérben élnek, és a tároló a felhasználó saját mappájába kerül. Biztonsági szempontból ez a szigorúbb változat; lemezhely miatt nem érdemes áttérni (lásd a 6. pontot). Ami eltér a fenti lépésektől:

1. **Telepítés és indítás.** A Docker CE csomagjai mellett a `docker-ce-rootless-extras` csomag és a disztribúció `uidmap` , `slirp4netns` (régebbi kernelnél `fuse-overlayfs`) csomagjai kellenek; a démont az üzemeltető felhasználó állítja be és futtatja:

```
dockerd-rootless-setuptool.sh install
systemctl --user enable --now docker
sudo loginctl enable-linger "$(id -un)"
# a démon kijelentkezés után is fusson
echo "export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock" >> ~/.bashrc
# a kliens ehhez a démonhoz beszéljen
```

A `docker` csoporttagság itt nem kell és nem számít (a 2. fejezet erre vonatkozó figyelmeztetései nem érvényesek, mert a démon nem root); a `docker compose` parancsokat ugyanez a felhasználó adja ki, a `docker context ls` mutatja, melyik démonhoz beszél a kliens. Szolgáltatáskezelés: `systemctl --user status|restart docker`, a démon naplója `journalctl --user -u docker`. A kiadás konténerei (`cap_drop: ALL`, `no-new-privileges`, csak olvasható gyökér, `tmpfs`) változtatás nélkül futnak.

2. **Portok.** Az 1024 alatti portot (443, 80) a nem root démon alapból nem kötheti: vagy `sudo sysctl -w net.ipv4.ip_unprivileged_port_start=0` (tartósan egy fájlban a `/etc/sysctl.d/` alatt), vagy magasabb port (`OPENHDID_HTTPS_PORT=8443`, `OPENHDID_HTTP_PORT=8080`) egy előtét vagy tűzfal-átirányítás mögött.
3. **A konténer felhasználója a gazdagépen (subuid, subgid).** A konténer `0`-s (root) felhasználója a gazdagépen az üzemeltető felhasználó; minden más konténerbeli `N` azonosító a `/etc/subuid` és `/etc/subgid` fájlban a felhasználónak kiosztott tartomány `kezdő + N - 1` eleme. Az alkalmazás a `82`-es felhasználóval fut, tehát a gazdagépen a `kezdő + 81` azonosító az övé:

   ```
   grep "^$(id -un):" /etc/subuid /etc/subgid     # például app:165536:65536
   echo $(( 165536 + 82 - 1 ))                     # 165617: ez a konténer 82-
   es felhasználója a gazdagépen
   docker compose exec web id                      # a konténer felől:
   uid=82(www-data)
   ```

   A `sudo chown 82:82` ezért rootless alatt egy idegen fiókhoz rendeli a fájlt, amelyet a konténer nem tud olvasni („Permission denied” a kulcsra, a `web` nem indul). A tartomány `kezdő` értéke gépenként más (gyakori a `100000` vagy a `165536`), mindig a saját sor számít; a `subgid` tartomány ugyanígy értendő a csoportra.
4. **Tanúsítvány és kulcs.** A fájlokat az üzemeltető másolja a `tls` mappába (a konténer felől ekkor a root tulajdona), majd a tulajdonost a konténer névterén belül írja a 82-es felhasználóra; ehhez a `CHOWN` képesség kell, mert a kiadás konténerei minden képességet eldobnak:

   ```
   cp /utvonal/fullchain.pem /utvonal/privkey.pem tls/
   docker compose run --rm --no-deps --user 0 --cap-add CHOWN -v "$PWD/tls":/t
   --entrypoint sh web \
     -c 'chown 82:82 /t/fullchain.pem /t/privkey.pem && chmod 444
   /t/fullchain.pem && chmod 400 /t/privkey.pem'
   ls -ln tls                                      # a gazdagépen a leképezett
   szám (például 165617) a helyes
   docker compose run --rm --no-deps web tls-check # futó veremnél: docker
   compose exec web openhdid tls-check
   ```

   A `--user 0` itt az üzemeltető saját felhasználója, nem a gazdagép rootja. Tanúsítványcsere után `docker compose exec web openhdid reload`. A `tls` és a `ca` mappa maradjon az üzemeltetőé, 755-tel; a `ca` fájljai 444-gyel olvashatók, tulajdonosváltás nélkül. A `backup.sh` a kulcsot és a `storage` kötetet a konténer saját felhasználójával menti és állítja vissza, így rootless alatt is a konténer által látott tulajdonos áll vissza, root nélkül.

5. **A hívók IP-címe.** A rootless démon alapértelmezett port-továbbítója (`builtin`) a kapcsolatokat a saját címéről adja tovább, így a konténer minden kérést az átjáró címéről lát: az access log `ip` mezője, az IP-alapú forgalomkorlát és az auditnapló címe hamis lenne. Két megoldás: a forrás-IP-t megtartó port-továbbító a démon felhasználói systemd-egységében (`~/.config/systemd/user/docker.service.d/override.conf` , benne `[Service]` és `Environment="DOCKERD_ROOTLESS_ROOTLESSKIT_PORT_DRIVER=slirp4netns"` , majd `systemctl --user daemon-reload && systemctl --user restart docker` ; a Docker dokumentációja szerint újabb kiadásokon a `pasta` hálózati illesztő az `implicit` port-továbbítóval ugyanezt gyorsabban adja), vagy egy TLS-t lezáró előtét a gazdagépen, `OPENHDID_TLS=off` és `TRUSTED_PROXIES` beállítással (6.5). Ellenőrzés: a saját gépéről küldött kérés az access logban a saját címével jelenjen meg, ne a `10.0.2.x` vagy `172.x` átjáróval.
6. **Tároló és lemez.** Az image-ek és a kötetek (az adatbázisé is) a `~/.local/share/docker` alá kerülnek (`docker info --format '{{.DockerRootDir}}'`); ha ez a gyökér-fájlrendszeren van, ugyanaz a lemez telik, mint a `/var/lib/docker` esetében, a hely tehát nem a rootless mód miatt lesz több. Rootful telepítés áttérése: `backup.sh` mentés a régi veremből, annak leállítása, a rootless démon alatt `docker compose up -d` , majd `backup.sh restore` (7. fejezet az üzemeltetési kézikönyvben); a két démon nem látja egymás image-eit és köteteit.

---

4 A beállítások működése a kész csomagban

A beállítások két rétegben élnek.

- **Telepítési értékek:** címek, jelszavak, illesztők, ütemezés, hálózat. Környezeti változók a `.env` fájlban.
- **Üzleti paraméterek:** csomaglisták, azonosítási határok, arculat, adatmegőrzés, belépési módok. A felületen, az *Adminisztráció › Rendszer › Beállítások* alatt szerkeszthetők, az adatbázisban tárolódnak.

Ez a szétválasztás a konténerben is ugyanaz, mint a hagyományos telepítésnél.

1. **Átadás.** A Compose a `.env` minden sorát környezeti változóként adja át a konténereknek (`env_file`). Ugyanebből a fájlból olvassa a saját változóit is (`OPENHDID_VERSION` , portok, profilok). A `db.env` -t csak a `mariadb` és a `migrate` szolgáltatás kapja meg. A konténeren belül nincs `.env` fájl, a Laravel a folyamat környezetét olvassa.
2. **Biztonságos alapértékek.** Az image-ben beállított alapértékek: `APP_ENV=production` , `APP_DEBUG=false` , `LOG_CHANNEL=stderr` . Amit a `.env` nem ad meg, az ezekkel indul.
3. **Titkok fájlból.** Bármely változó megadható fájlként is: `NÉV_FILE=/run/secrets/…` . Indításkor a `NÉV` változó a fájl tartalmát kapja; ha a kettőt egyszerre adja meg, az hiba. Docker- és Kubernetes-titkokhoz ajánlott, mert a sima környezeti változót a `docker inspect` megmutatja. Ugyanez érvényes a beépített MariaDB fiókjaira: a `db.env` -ben a `MARIADB_ROOT_PASSWORD_FILE` és a migrációs fiók `DB_PASSWORD_FILE` (vagy `DB_USERNAME_FILE`) kulcsa, a `.env` -ben a futási fiók `DB_PASSWORD_FILE` kulcsa használható; az első induláskor a fiókkészítő szkript a fájlok tartalmával hozza létre a fiókokat. A titokfájlt ekkor a `mariadb` és a `migrate` konténerbe is csatolni kell ugyanazon az útvonalon (Compose `secrets` vagy kötet), nem csak az alkalmazás-konténerekbe.

4. **Ellenőrzés induláskor.** Minden szerepkör indulásakor az indítószkript megnézi a kötelező értékeket, és hibás beállítással a konténer érthető hibaüzenettel leáll (`docker compose logs`). Hibának számít:
   - az érvénytelen `APP_KEY`;
   - ha az `APP_URL` nem https;
   - a hiányzó adatbázis-adat;
   - az ismeretlen `APP_ENV` érték: csak `production`, `staging`, `demo`, `local` vagy `testing` lehet, mert például a `prod` vagy a `Production` az éles védelmeket csendben kikapcsolná;
   - élesben (`APP_ENV=production`) az `APP_DEBUG=true`, a mindenkit megbízhatónak vevő `TRUSTED_PROXIES` (`*`, `**`, `0.0.0.0/0`, `::/0`), a `SESSION_SECURE_COOKIE=false`, az üres `DB_PASSWORD` (socketes kapcsolat kivételével), és ha a gyorsítótár, a sor vagy a munkamenet Redis-t használ, az üres `REDIS_PASSWORD`. A konténer ilyenkor 64-es kilépési kóddal áll meg, és a `restart` szabály miatt újraindulgat; az ok a `docker compose logs web` kimenetében. Részletes hibaüzenetekhez az üzemeltetési kézikönyv 5. fejezetének hibakeresési receptje való, nem a `production` környezet debug módja.

   Hibának számít még: az érvénytelen vagy túl tág `HDID_RATE_LIMIT_EXEMPT_NETWORKS`-elem (az elem megnevezésével), a hibás `OPENHDID_NGINX_*`, `OPENHDID_FPM_MEMORY_LIMIT`, `OPENHDID_FPM_MAX_CHILDREN` és `OPENHDID_FPM_BACKLOG` érték, és élesben a demóadat az adatbázisban (a `migrate` áll meg, 13. fejezet). Csak figyelmeztetés jár a naplóba író levélküldésért és SMS-ért, az önaláírt tanúsítványért, a `MAIL_REQUIRE_TLS=false` beállításért, az üres `REDIS_SESSION_HOST`-ért (a munkamenetek a fő Valkey-ben vannak), azért, ha a PHP-FPM folyamatok és a memóriakorlátjuk összege nem fér a `web` konténer memóriaplafonjába, az `OPENHDID_NGINX_RATE_LIMIT=0` beállításért, és ha `OPENHDID_TLS=on` mellett a `TRUSTED_PROXIES` egy proxyt nevez meg (az nginx ekkor a proxy címét látja, a kéréskorlát minden mögötte álló klienst egynek számol), illetve `off` mellett egyet sem.
5. **Gyorsítótár.** Az ellenőrzés után a konfiguráció, az útvonalak és az események gyorsítótára a memóriában lévő `/tmp`-be épül. A titkokat is tartalmazó konfigurációs gyorsítótár így lemezre sem kerül. A PHP-FPM dolgozói ezt olvassák, a konténer környezetét nem látják (`clear_env`).
6. **Módosítás:** a `.env` szerkesztése után `docker compose up -d`. A Compose az érintett konténereket újraindítja, a gyorsítótár újraépül; a hagyományos `php artisan optimize` és `queue:restart` itt nem kell. A felületi beállítások azonnal élnek, újraindítás nélkül.

> **APP_KEY**
>
> Soha nem generálódik magától, és meglévő telepítésen nem cserélhető: cseréje a titkosított adatok, a kulcsolt keresőlenyomatok és az ügyféltörzs tokenjeinek elvesztésével jár. A mentés része legyen, a mentéstől elkülönített hozzáféréssel.

---

5 Környezeti változók

A teljes lista magyar magyarázattal a kiadás `env.example` fájljában található. A legfontosabbak:

> **Backslash az értékben.**
>
> Tartományi felhasználónevet (`DOMAIN\user` , például az ügyféltörzs vagy az NTLM-es Exchange fiókja) idézőjel nélkül vagy aposztrófok között írjon: `EMD_SYNC_USERNAME=DOMAIN\user` vagy `EMD_SYNC_USERNAME='DOMAIN\user'` . Dupla idézőjelben a Compose a `\n` és `\r` sorozatot sortöréssé alakítja, így a `"DOMAIN\nagy"` két sorra esik. Az alkalmazás a nevet JSON-ban, helyesen kódolva küldi, a backslash-t nem kell kettőzni.

Kiadás, hálózat, konténer

KULCS	ALAPÉRTÉK	JELENTÉS
`OPENHDID_VERSION`	–	A futtatott image verziója. A letöltött `env.example` -ben a kiadás száma áll, a kiadási folyamat írja be; élesben mindig pontos verziószám, ne `latest` . Frissítésnél ezt kell átírni (14. fejezet).
`COMPOSE_PROFILES`	`db,redis`	Választható szolgáltatások: `db` beépített MariaDB, `redis` beépített Valkey (alapból mindkettő). Külső adatbázisnál és Valkey-nél (10k profil) üres, a `DB_HOST` és a `REDIS_HOST` a külső kiszolgálóra mutat; a 10k profil `primary` értéke az ütemezőt és az ügyféltörzs-feldolgozót indítja az elsődleges gépen.
`COMPOSE_FILE`	`compose.yaml:profiles/5k/compose.yaml`	A használati profil kiegészítő fájlja a `compose.yaml` mellett (15. fejezet): `profiles/5k` vagy `profiles/10k` . A profil adja a `HDID_SIZING_PROFILE` értékét, a PHP-FPM folyamatszámokat, a feldolgozók számát és a beépített MariaDB memóriáját; a `.env` azonos nevű sorai felülírják.
`HDID_SIZING_PROFILE`	a profil fájlja adja	A profil neve (`5k` , `10k` , `20k`) a Rendszerállapot „Kapacitás” kártyájának, amely a mért aktív fiókok csúcsát a profil tervezett egyidejű felhasználószámához méri, és 80 %-nál sárgul. Profil nélkül a kártya csak tájékoztat.
`OPENHDID_NFS_SERVER` , `OPENHDID_NFS_PATH`	üres	Csak a 10k profilnál: a két alkalmazásgép közös `storage` kötetének NFS-kiszolgálója és exportált útvonala (15.4 pont).
`OPENHDID_BIND_IP`	`0.0.0.0`	A gazdagép címe, amelyen a 80-as és 443-as port figyel (6.1 pont).
`OPENHDID_HTTPS_PORT` , `OPENHDID_HTTP_PORT`	`443` , `80`	A gazdagép portjai; a konténerben mindig 8443 és 8080.

KULCS	ALAPÉRTÉK	JELENTÉS
`OPENHDID_TLS`	`on`	`on` saját tanúsítvány, `off` terheléselosztó mögött, `selfsigned` csak kipróbáláshoz.
`OPENHDID_TLS_CERT` , `OPENHDID_TLS_KEY`	`/etc/openhdid/tls/…`	A tanúsítvány és a kulcs helye a konténerben.
`OPENHDID_EXTRA_HOSTS`	üres	További kiszolgálónevek az `APP_URL` mellett, vesszővel; az nginx és az alkalmazás is fogadja őket (6.2 pont). Gépi hívásra valók (telefonközpont, mobil háttérrendszer): minden hivatkozás, űrlap és átirányítás az `APP_URL` -lel készül, ezért a panel és a portál böngészőből csak az `APP_URL` címén használható.
`OPENHDID_HTTP_REDIRECT`	`true`	A 80-as porton a böngészős GET/HEAD 308-cal a HTTPS-címre megy; `false` : minden elutasítva.
`OPENHDID_MONITORING_ALLOW`	üres	A `/health` és `/up` elérésére jogosult címek (IP vagy CIDR).
`OPENHDID_NGINX_WORKERS`	üres (a profil adja: profil nélkül 2, 5k és 10k: 4)	nginx-dolgozók száma; a web memóriaplafonja ezzel számol (dolgozónként kb. 80 MB). `auto` = processzormagonként egy: több magos gépen az `OPENHDID_WEB_MEM_LIMIT` -et is emelni kell, különben induláskor figyelmeztetés jön.
`OPENHDID_FPM_PM` , `_MAX_CHILDREN` , `_START_SERVERS` , `_MIN_SPARE_SERVERS` , `_MAX_SPARE_SERVERS`	a profil adja (profil nélkül `dynamic` , 8, 2, 2, 4; 5k: `dynamic` , 12, 4, 4, 8; 10k: `static` , 12)	PHP-FPM folyamatkezelés webkonténerenként. A képbe épített `php.ini` a `max_execution_time` értékét 300 mp-re állítja (FPM és CLI egyaránt); a megszakított webes kérés folyamata legfeljebb 125 mp után felszabadul (`request_terminate_timeout`). Ha a folyamatszámot emeli, a `OPENHDID_WEB_MEM_LIMIT` -et is emelni kell (lent), különben a konténer induláskor és a Kapacitás kártya figyelmeztet.
`OPENHDID_FPM_MEMORY_LIMIT`	`128M`	Egy webes kérés memóriakorlátja (legalább 64M). Az importok és az átvétel a feldolgozóban 256M-mel futnak. Ha egy panelműveletnél „Allowed memory size” hiba jelenik meg: `256M` , nagyobb `OPENHDID_WEB_MEM_LIMIT` mellett.

KULCS	ALAPÉRTÉK	JELENTÉS
`OPENHDID_FPM_BACKLOG`	`64`	Ennyi kérés várhat szabad PHP-FPM folyamatra; a sor betelte után az nginx azonnal 503-at ad `Retry-After: 10` fejléccel, ahelyett, hogy a kérés percekig várna.
`OPENHDID_WEB_MEM_LIMIT`	profil nélkül `1920m`, 5k és 10k: `2688m`	A `web` konténer memóriaplafonja: PHP-FPM folyamat × (`OPENHDID_FPM_MEMORY_LIMIT` + kb. 22 MB) + 208 MB OPcache + nginx-dolgozók × 80 MB + 256 MB `/tmp` + 64 MB.
`OPENHDID_DB_BUFFER_POOL`, `OPENHDID_DB_MAX_CONNECTIONS`, `OPENHDID_DB_MEM_LIMIT`	a profil adja (profil nélkül `256M`, 25, `1g`; 5k: `1G`, 40, `2g`)	A beépített MariaDB puffere, kapcsolatszáma és memóriaplafonja; a plafon a puffer + 192 MB + kapcsolat × 20 MB, a kapcsolatszám legalább minden PHP-FPM folyamat + a feldolgozók + 1 átvétel + 3 ütemező + 6 tartalék.
`OPENHDID_VALKEY_MAXMEMORY`, `OPENHDID_VALKEY_MEM_LIMIT`	`256mb`, `768m`	A beépített fő Valkey memóriakorlátja és konténerplafonja (a plafon kb. 2 × a korlát + 256 MB, az AOF-újraírás miatt). Betelt tárnál az írás hibát ad, kulcs nem vész el; a Rendszerállapot „Valkey / Redis” kártyája 80 % fölött sárga. Sok várakozó üzenetnél emelendő.
`OPENHDID_VALKEY_SESSIONS_MAXMEMORY`, `OPENHDID_VALKEY_SESSIONS_MEM_LIMIT`	profil nélkül `128mb`, `384m`; 5k: `384mb`, `896m`	A beépített munkamenet-Valkey (`valkey-sessions`) korlátja és plafonja (kb. 2 × a korlát + 128 MB). Betelt tárnál a legrégebben használt munkamenet szorul ki; nagy támadáskor a régóta tétlen felhasználóknak újra be kell lépniük.
`OPENHDID_NGINX_RATE_LIMIT`, `OPENHDID_NGINX_RATE_BURST`	`20`, `100`	Az alkalmazás felé menő kérések (a statikus fájlok nem) száma másodpercenként kliens-címenként, és ennyivel több egyszerre; felette 429 (6.8 pont). A `HDID_RATE_LIMIT_EXEMPT_NETWORKS` címeire nem hat (egy NAT mögötti ügyfélszolgálat oda kerül, nem a korlátot kell emelni). `0` kikapcsolja (csak mérő- vagy tesztpéldányon).
`OPENHDID_NGINX_CONN_LIMIT`	`16`	Egyidejű kérések száma kliens-címenként az alkalmazásban; felette 429. `0` kikapcsolja.
`OPENHDID_NGINX_CLIENT_TIMEOUT`	`15`	A kéréstörzs, a küldés és a keepalive időkorlátja másodpercben (5–300); a fejlécre 10 mp jut. A lassú klienst így nem tartja fel a kapcsolat.

KULCS	ALAPÉRTÉK	JELENTÉS
`HDID_RATE_LIMIT_EXEMPT_NETWORKS`	üres	Vállalati hálózatok (IPv4/IPv6-cím vagy CIDR, vesszővel vagy szóközzel; elemenként legfeljebb /8, illetve /32), amelyek mentesülnek a kliens-cím szerinti korlátok alól: az nginx kérés- és kapcsolatkorlátja, a belépési és céges belépési útvonalak és a többi vendégútvonal címenkénti korlátja, a panel belépési korlátja. Nem mentesít: az e-mail-címenkénti és számonkénti korlátok, a rendszerszintű belépési üzenetplafonok, a fiókzárolás, a bejelentkezett felhasználó és a gépi kulcsok korlátjai. Érvénytelen vagy túl tág elemnél senki sem mentes, és a konténer nem indul. Terheléselosztó mögött csak `OPENHDID_TLS=off` és a `TRUSTED_PROXIES` mellett hat (a valódi kliens-cím kell); az elosztó címét felvenni tilos. NAT mögötti vállalati telepítésnél ez a javasolt megoldás a kéréskorlát emelése vagy kikapcsolása helyett; a konténer induláskor kiírja a mentes hálózatokat.
`OPENHDID_DEMO_DATA`	`false`	Demóadatok betöltése (13. fejezet), csak `APP_ENV=demo` mellett; más környezetben a `migrate` hibával megáll.

Alkalmazás

KULCS	ÉRTÉK ÉLESBEN	JELENTÉS
`APP_ENV`, `APP_DEBUG`	`production`, `false`	Hibaválaszban nincs belső részlet; más kombinációval élesben nem indul.
`APP_KEY`	egyszer generált	Lásd 3. és 4. fejezet.
`APP_URL`	`https://hdid.example.org`	Pontosan a böngészőben használt cím: minden generált cím (belépési levél, SSO visszatérési cím, átirányítások, oldal-eszközök) ebből készül, nem a kérés fejléceiből, és a kiszolgálónév is ebből képződik. Más `Host` vagy `X-Forwarded-Host` fejlécre 400 a válasz („Ismeretlen kiszolgálónév”, API-n `reason: unknown_host`). Az `APP_URL`-ből és az `APP_KEY`-ből képződik a telepítésazonosító is (üzemeltetési kézikönyv 6. fejezet).
`APP_LOCALE`, `APP_TIMEZONE`	`hu`, `Europe/Budapest`	Alapnyelv; megjelenített idők és az ütemező időzónája.

KULCS	ÉRTÉK ÉLESBEN	JELENTÉS
`APP_MAINTENANCE_DRIVER`	`cache`	A karbantartási mód minden példányra hat.
`LOG_CHANNEL` , `LOG_LEVEL`	`stderr` , `warning`	Napló a konténer kimenetére (9. fejezet).
`DB_*`	futási fiók	`DB_HOST=mariadb` a beépített adatbázisnál; a `db.env` a migrációs fiókot adja.
`MYSQL_ATTR_SSL_CA`	üres	Külső adatbázis TLS-kapcsolatához a CA-fájl a konténerben (például `/etc/openhdid/ca/db-ca.pem`).
`SESSION_*`	`redis` , 120, titkosított, `__Host-openhdid_session` , Secure, HttpOnly, Lax	Munkamenet a munkamenet-Valkey-ben (`SESSION_CONNECTION=session`); 120 perc inaktivitás. `database` csak kipróbáláshoz, Valkey nélkül.
`REDIS_SESSION_HOST` , `REDIS_SESSION_PORT` , `REDIS_SESSION_PASSWORD` , `REDIS_SESSION_USERNAME` , `REDIS_SESSION_DB`	üres; a beépített Valkey mellett a `compose.yaml` `valkey-sessions` -t ad	A munkamenetek Valkey-je. A beépített Valkey-nél (`REDIS_HOST=valkey`) a `compose.yaml` magától a `valkey-sessions` szolgáltatásra irányítja; külső Valkey-nél (10k, 20k) egy második, `allkeys-lru` politikájú példány címe kerül ide. Üres kulcs a `REDIS_*` párját veszi át: ilyenkor a munkamenetek a fő Valkey-ben vannak, a konténer élesben figyelmeztet, a Munkamenet-tár kártya sárga.
`CACHE_STORE` , `QUEUE_CONNECTION`	`redis`	Gyorsítótár és üzenetsor a Valkey-ben (`REDIS_HOST=valkey` a beépítettnél, külső kiszolgáló a 10k és 20k profilnál; `REDIS_PASSWORD` élesben kötelező). Ez veszi le a munkamenetek, a zárak és a sor terhét az adatbázisról, és több webpéldányhoz nélkülözhetetlen.
`REDIS_PREFIX` , `CACHE_PREFIX`	telepítésenként egyedi, például `openhdid-eles-` , `openhdid-eles-cache-`	A kulcsok előtagja. A beépített Valkey-nél az alapérték csak tájékoztat, külső Valkey-nél sárga a Valkey-kártya. Csak üres sor mellett cserélhető (üzemeltetési kézikönyv 6. fejezet), és a cserével minden munkamenet véget ér.
`REDIS_DB` , `REDIS_CACHE_DB`	`0` , `1`	A fő Valkey adatbázisai (munkamenet tartalékként és sor, illetve gyorsítótár). A mentés visszaállítása és a `./backup.sh signout` a munkamenet- és a gyorsítótár-adatbázist teljesen üríti (`FLUSHDB`): egy adatbázisszámot csak egy telepítés használjon, a menedzselt Valkey engedje a `FLUSHDB` -t.

KULCS	ÉRTÉK ÉLESBEN	JELENTÉS
`HDID_PRIORITY_QUEUE`	`priority`	Az elsőbbségi sor a legfeljebb 15 percig érvényes belépési kódoknak (az ügyfél SMS-belépőkódja, a munkatársi második lépés kódja); a `worker` előbb ezt hallgatja. Az új `compose.yaml` kell hozzá (a régi `worker default` paranccsal a kódok nem mennek ki; UPGRADE.md).
`*_QUEUE_RETRY_AFTER`	1260	Nagyobb legyen az átvétel 1200 mp-es korlátjánál.
`TRUSTED_PROXIES`	`127.0.0.1`	Csak `OPENHDID_TLS=off` mellett: a terheléselosztó (Kubernetesen az ingress-vezérlő) címei. Az nginx csak ezektől veszi át a kliens címét, és a PHP felé minden továbbító fejlécet a saját értékére cserél (6.5 pont).
`SANCTUM_TOKEN_EXPIRATION`	43200	A mobilalkalmazás bejelentkezésének lejárata percben.
`HDID_CSP_ENABLED` , `HDID_CSP_EXTRA_SOURCES`	`true` , üres	Tartalombiztonsági szabályzat; csak indokolt, pontos források vehetők fel.
`HDID_PHONE_REGION`	`HU`	A hívószámok értelmezésének alapértelmezett országa.
`HDID_SOURCE_URL`	az image kiadására mutat	A „Forráskód” hivatkozás a portál és a panel láblécében; módosított változatnál a módosított forrásra mutasson (AGPL-3.0, 13. pont).
`SHOW_SC`	`true`	A „Forráskód” hivatkozás megjelenítése; `false` elrejti akkor is, ha a cím meg van adva. Az AGPL-3.0 13. pontja szerint a hálózati felhasználóknak ekkor más úton kell felajánlani a forrást.

Külső rendszerek

KULCSOK	JELENTÉS	
`HDID_HTTP_PROXY`	üres	Kimenő proxy az alkalmazás minden HTTP(S)-kéréséhez (ügyféltörzs, Exchange, Ozeki SMS, ADFS, Entra), a web-, a worker- és az ütemező-konténerből egyaránt. Alakja `http://proxy.example.org:3128`, jelszóval `http://felhasznalo:jelszo@proxy.example.org:3128`; a felhasználónév és a jelszó különleges karakterei %-kódolva (`@` = `%40`, `$` = `%24`, `#` = `%23`). Windows-hitelesítésű (NTLM) proxy nem támogatott. Üresen minden kérés közvetlenül megy. Hibás címmel a konténer nem indul; a használt proxyt (jelszó nélkül) a Rendszerállapot „Kimenő proxy” kártyája mutatja. Az SMTP, az adatbázis és a Valkey nem HTTP, rájuk nem hat; az image letöltésére sem (az a Docker-démon proxyján megy).
`HDID_NO_PROXY`	üres	A proxy nélkül elért kiszolgálók vesszővel: név (az aldomaineket is lefedi, például `belso.example.org` vagy `.belso.example.org`), IP-cím vagy tartomány (`10.0.0.0/8`), `*` = mind. A `localhost`, a `127.0.0.0/8` és a `::1` mindig közvetlen. Ide kerülnek a belső kiszolgálók: tipikusan az ügyféltörzs, az ADFS, a helyi Exchange és az SMS-átjáró.
`MAIL_MAILER`	`ews` / `smtp` / `log`	Levelezési illesztő: `ews` az Exchange webszolgáltatás, `smtp` SMTP-kiszolgáló. A `log` csak naplóba ír, élesben a rendszerállapot figyelmeztet. Próba: `hdid:test-mail cím --now` (11. fejezet). EWS-en a levelek mentett másolat nélkül mennek (a titkot hordozó levél nem marad meg a küldő postafiók Elküldött elemei mappájában); másolat csak a kifejezetten nem titkos levélről (például próbalevél) marad.
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	feladó cím és név	EWS-nél a feladó postafiók legyen az EWS-fiók, vagy annak legyen küldési joga rá.

KULCSOK	JELENTÉS	
`EWS_URL`, `EWS_AUTH`, `EWS_USERNAME`, `EWS_PASSWORD`	`https://…/EWS/Exchange.asmx`; `basic` / `ntlm` / `oauth`	Az Exchange webszolgáltatás címe és hitelesítési módja; `basic` és `ntlm` esetén a postafiók felhasználója és jelszava. Tartományi felhasználónév (`DOMAIN\user`) idézőjel nélkül vagy aposztrófok között írandó. Ha az Exchange csak Windows-hitelesítést kínál (a próbaküldés hibája `HTTP 401 … the server offers: NTLM`), `EWS_AUTH=ntlm` kell: az image 1.8.0-től saját, NTLM-képes libcurl-t hoz (az Alpine csomagja NTLM nélkül épül), amit az indításkori ellenőrzés és a Rendszerállapot Levelezés kártyája is számon kér. Alternatíva a basic engedélyezése az EWS virtuális könyvtáron, vagy küldés SMTP-összekötőn (`MAIL_MAILER=smtp`, `MAIL_HOST`, `MAIL_PORT=587`, STARTTLS).
`EWS_OAUTH_TENANT`, `EWS_OAUTH_CLIENT_ID`, `EWS_OAUTH_CLIENT_SECRET`, `EWS_OAUTH_SCOPE`	Entra-regisztráció adatai; a hatókör alapból `https://outlook.office365.com/.default`	`EWS_AUTH=oauth` esetén: a bérlő GUID-ja, az alkalmazás kliensazonosítója és titka, a postafiókra adott EWS-jogosultsággal. A hatókört csak eltérő felhő esetén kell megadni.
`EWS_VERIFY_TLS`	`true`	Maradjon `true`; belső tanúsítványkiadóval kiállított Exchange-tanúsítványnál a CA a `ca` mappába kerül.
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_SCHEME`	SMTP-kiszolgáló adatai; `smtp` vagy `smtps`	`MAIL_MAILER=smtp` esetén. `smtps` közvetlen TLS (jellemzően 465-ös port); `smtp` vagy üres: STARTTLS (587), amely kötelező (`MAIL_REQUIRE_TLS`).
`MAIL_REQUIRE_TLS`	Alapból `true`: SMTP-n a STARTTLS kötelező; ha a kiszolgáló nem kínálja, a küldés teendőt mondó hibával megáll (a jelszó és a levél nem megy ki titkosítatlanul), és a Levelezés kártya piros. Üres vagy hibás érték is `true`. `false` csak titkosítás nélküli helyi próbakiszolgálóhoz; ekkor a kártya sárga, és a konténer élesben figyelmeztet. EWS-re és `smtps`-re nem vonatkozik.	
`SMS_DRIVER`	`ozeki` / `log`	SMS-küldés az Ozeki NG átjárón; a `log` csak naplóba ír, élesben a rendszerállapot hibát jelez. Próba: `hdid:test-sms +36… --now`.
`OZEKI_URL`, `OZEKI_USERNAME`, `OZEKI_PASSWORD`	az átjáró API-címe (például `https://sms.example.org:9501/api`), felhasználó, jelszó	A rendszer POST-kérést küld, a hozzáférés nem kerül a címsorba; csak kapcsolódási hibánál ismétel.

KULCSOK	JELENTÉS	
`SMS_DELIVERY_CHECKS`	üres (`30,120,300,1800,7200`)	Az SMS kézbesítési jelentésének lekérdezései a küldés után ennyi másodperccel, vesszővel, legfeljebb öt időpont, mindegyik 1 és 86 400 között; hibás érték mellett az alapérték érvényes, a Rendszerállapot figyelmeztet.
`OZEKI_STATUS_ACTION` , `OZEKI_DELIVERED_STATUSES` , `OZEKI_UNDELIVERED_STATUSES`	üres (`querymessage` ; `delivered,deliveredtohandset` ; `undelivered,notsent,deliveryfailed,failed,rejected,expired`)	Az Ozeki állapot-lekérdezésének API-művelete és az átjáró állapotszavai (kézbesítve, illetve nem kézbesíthető); az átjáró verziójától függ, a saját átjárón ellenőrizendő (11. fejezet).
`EMD_SYNC_DRIVER` , `EMD_SYNC_API_URL`	`spreadsheet` , teljes export-URL	Átvételi mód: hitelesített POST → CSV/XLSX fájl az export végpontjáról; az export kéréstörzse (JSON) a panel *Beállítások › Ügyféltörzs (EMD)* fülén, csak SuperAdmin szerkeszti. A régi GET JSON API-hoz `json` , `EMD_SYNC_API_TOKEN` (statikus Bearer-token) és `EMD_SYNC_API_TIMEOUT` (60 mp).
`EMD_SYNC_LOGIN_URL` , `EMD_SYNC_REFRESH_URL` , `EMD_SYNC_USERNAME` , `EMD_SYNC_PASSWORD`	az ügyféltörzs belépési és tokenfrissítési címe, belépési adatai	Belépés: POST JSON `{Password, Username}` ; tokenfrissítés: POST JSON `{refreshToken}` . Mindkét válasz `access_token` és `refresh_token` ; a `roles` mezőt a rendszer figyelmen kívül hagyja. Sikertelen tokenfrissítés után egyszeri új belépés következik. Tartományi felhasználónév (`DOMAIN\user`) idézőjel nélkül vagy aposztrófok között írandó.
`EMD_SYNC_AUTH_TIMEOUT` , `EMD_SYNC_EXPORT_TIMEOUT`	20 / 600 másodperc	Belépés és fájlletöltés külön időkorláttal; nem a token érvényessége. A teljes háttérfeladat korlátja 1200 mp, a letöltési időkorlát ennél kisebb legyen.

KULCSOK	JELENTÉS	
`EMD_SYNC_ID_LIST_URL`	üres	Az ID-lista (például a szervezetek) lekérdezésének címe: POST az exporttal azonos tokennel, a Beállításokban megadott kéréstörzzsel, az `EMD_SYNC_EXPORT_TIMEOUT` időkorlátjával. Csak akkor kell, ha az export kéréstörzse a `{{ ids }}` helyőrzőt használja; üresen az ID-lista ki van kapcsolva. A helyőrző idézőjelek között szövegként (`"ids": "{{ ids }}"` → `"3,12"`), idézőjel nélkül, szögletes zárójelek között JSON-listaként (`"ids": [{{ ids }}]` → `[3,12]`) töltődik ki; az ügyféltörzs hibaválaszának törzse a `docker compose logs worker` kimenetében olvasható.
`EMD_SYNC_EXPORT_FORMAT` , `EMD_SYNC_CSV_DELIMITER`	`xlsx` , `,`	A várt exportfájl `xlsx` vagy `csv` ; a CSV-elválasztó egyetlen karakter. A CSV-kódolás a *Beállítások › Ügyféltörzs (EMD)* fülön állítható.
`EMD_SYNC_TOKEN_KEEP_ALIVE`	`true` / `false` (`on` / `off` , `1` / `0` is)	Az ügyféltörzs API-tokenjének 15 percenkénti háttérfrissítése; kikapcsolva az átvétel szükség szerint frissít, sikertelen frissítés után egyszer újra belép. Olvashatatlan érték esetén az alapérték érvényes. Módosítás után `docker compose up -d` .
`EMD_SYNC_QUEUE`	`sync`	A panelről indított átvétel sora; a `sync-worker` szolgáltatás dolgozza fel, így a legfeljebb 20 perces átvétel nem tartja fel az e-maileket és SMS-eket. Ha várakozó átvételt öt percen túl egyetlen feldolgozó sem vesz fel, a rendszerállapot figyelmeztet.
`EMD_SYNC_CRON`	`0 */3 * * *`	Teljes ügyféltörzs-átvétel 3 óránként; üresen csak kézzel. A 15 percenkénti tokenfrissítés ettől független. A régi `SYNC_*` nevek álnévként működnek; kitöltött új név elsőbbséget kap.
`CALLCENTER_*`	Örökölt, rögzített telefonközponti kulcs és aláírás. Új partnerhez a felületen kezelt API-kulcs ajánlott; a `CALLCENTER_SIGNATURE_TTL` (300) az aláírt kérés időbélyegének megengedett eltérése.	
`USER_*` , `CLIENT_*` (`ADFS` , `ENTRA`)	Négy alkalmazás-regisztráció az egyszeri bejelentkezéshez; az Entra-bérlő csak GUID lehet.	

KULCSOK	JELENTÉS
`USER_ADFS_EMAIL_CLAIM`, `CLIENT_ADFS_EMAIL_CLAIM`	Az ADFS-tokenek igényei (claimek), amelyeket a rendszer sorrendben a fiók e-mail-címével összevet, vesszővel; üresen `emailaddress,email,upn,preferred_username,unique_name`.
`USER_ADFS_SCOPES`, `CLIENT_ADFS_SCOPES`	Az `openid profile email` mellé kért további hatókörök (például `allatclaims`); nem kötelező, üresen nincs.
`USER_ADFS_RESOURCE`, `CLIENT_ADFS_RESOURCE`	A Web API azonosítója `resource` paraméterként (ennek a kiadási szabályai kerülnek az access tokenbe); nem kötelező.
`CLIENT_ENTRA_TENANTS`	Beállítás után a *Beállítások › Ügyfélportál › Belépési módok* blokkban a „Céges belépés: Microsoft Entra ID” kapcsoló alatt ellenőrizhető a bérlők és domainjeik állapota (ügyféltörzsbeli tételszámmal, színes jelvénnyel). Az ügyfélportál több Entra-bérlője: GUID-ok vesszővel, bérlőnként választható `:domain|domain` szűrővel (domain nélkül a bérlő megbízható); több bérlőnél a közös `organizations` végpont, a token bérlője a listához kötve. Egy bérlőnél elég a `CLIENT_ENTRA_TENANT`; mindkettő kitöltve az egyes bérlő a lista része, a Rendszerállapot sárga (11. fejezet).
`USER_SSO_MATCH`, `CLIENT_SSO_MATCH`	A céges belépés tokenjéből melyik igény találja meg a fiókot: `email` (alapérték) vagy `employee_id` (a dolgozói azonosító, az ügyféltörzstétel külső azonosítójával összevetve; a token e-mail-címe ekkor nem számít). Ismeretlen érték: a szolgáltató „nincs beállítva”. Módosítás után `docker compose up -d`.
`USER_SSO_EMPLOYEE_ID_CLAIM`, `CLIENT_SSO_EMPLOYEE_ID_CLAIM`	Dolgozói azonosító módban az igény neve; üresen `employeeId`. ADFS-nél az igénytípus-URI végződése is jó, az ID token után az access token is számít.
`HDID_SSO_REQUIRE_SUBJECT`	Alapból `true`: a szolgáltató állandó azonosítója (Entra `oid`, ADFS `sub`) nélkül érkező token elutasul. `false` esetén az ilyen belépés kötés nélkül megy, az auditnapló jelzi.

KULCSOK	JELENTÉS
`CLIENT_LOGIN_REQUIRE_EMD`	Alapból `true`: az ügyfél csak az e-mail-címét viselő, jelen lévő ügyféltörzstétellel léphet be. `false` esetén tétel nélkül, hiányzó tétellel vagy eltérő címmel is beléphet; a lezárás és a csomagjogosultság marad, munkatársra nem hat, a Rendszerállapot sárgán jelzi. Módosítás után `docker compose up -d`.
`HDID_MFA_PROVIDER_AMR`	Kétlépcsős munkatársi belépésnél a céges szolgáltató MFA-ját igazoló további `amr` értékek, vesszővel (beépítve: Entra `mfa`, ADFS `http://schemas.microsoft.com/claims/multipleauthn`); csak bekapcsolt „A céges szolgáltató MFA-jának elfogadása” beállítás mellett számít. ADFS-nél az igényt az `authnmethodsreferences` kiadási szabály adja.
`HDID_ADFS_DEBUG`	`true` esetén minden ADFS-belépés a kérést, a tokenválaszt, a két token fejlécét és tartalmát az érkezett alakban és visszafejtve (személyes adatokkal) a naplóba írja (`docker compose logs web`); az aláírásnak csak a hossza látszik, a frissítő token nem kerül bele. Alapértéke `false`; amíg be van kapcsolva, a „Belépési szolgáltatók” csempe sárga.

6 Hálózat, kiszolgálónév és HTTPS

6.1 Figyelő cím és portok

A konténerben az nginx mindig ugyanazokon a nem privilegizált portokon figyel: 8080 (HTTP) és 8443 (HTTPS), minden konténercímen. Ezekhez nem kell hozzányúlni.

Hogy a gazdagép melyik címén és portján érhető el a szolgáltatás, azt a Compose port-leképezése dönti el a `.env` alapján:

```
OPENHDID_BIND_IP=192.0.2.10    # egyetlen hálózati kártya címe; 0.0.0.0 = minden IPv4-cím
OPENHDID_HTTPS_PORT=443
OPENHDID_HTTP_PORT=80
```

Ha a 80-as portra egyáltalán nincs szükség, a `compose.yaml` `web` szolgáltatásából törölheti a 8080-as leképezést, vagy az `OPENHDID_HTTP_REDIRECT=false` minden HTTP-kérést elutasít. A 443-as porton a tűzfal engedje a klienseket, a telefonközpontot és a mobil kiszolgálót; az adatbázis és a Valkey portja nincs kivezetve a gazdagépre.

6.2 Kiszolgálónév (domain)

- **A kiszolgálónév** az `APP_URL` hostjából képződik; ez az egyetlen hely, ahol meg kell adni. További nevek az `OPENHDID_EXTRA_HOSTS` változóban adhatók meg (a tanúsítványnak ezeket is le kell fednie); ezek gépi hívásra valók, böngészőből a panel és a portál csak az `APP_URL` címén működik.
- **Az alkalmazás is ellenőriz:** csak az `APP_URL` gépnevére, a konténer belső életjel-próbájának `localhost` nevére és az `OPENHDID_EXTRA_HOSTS` neveire szóló kérést fogadja, másra (például egy hamis `X-Forwarded-Host` fejlécre) 400 „Ismeretlen kiszolgálónév” a válasz; minden generált cím az `APP_URL`-ből készül.
- **Ismeretlen Host fejléc vagy SNI:** a TLS-kézfogás elutasítva, HTTP-n kapcsolatbontás (444). Idegen név alatt az alkalmazás nem szolgálható ki.
- **A 80-as porton** csak a böngészős GET/HEAD kérés kap 308-as átirányítást az `APP_URL` címére. A gépi (`/api/`) és hitelesítési (`/auth/`) útvonalak, illetve minden más metódus kapcsolatbontást kap.
- **Az nginx-konfigurációt** az indítószkript minden induláskor a környezetből állítja elő, csak ellenőrzött értékekkel, majd `nginx -t` próbát futtat. Kézi szerkesztésre nincs szükség.

6.3 HTTPS saját tanúsítvánnyal (TLS 1.3)

Alapértelmezés (`OPENHDID_TLS=on`): a `tls/fullchain.pem` (a kiszolgáló tanúsítványa, utána a köztes tanúsítványok) és a `tls/privkey.pem`. Induláskor a rendszer ellenőrzi:

- a kulcs a tanúsítványhoz tartozik-e;
- a tanúsítvány lefedi-e az `APP_URL` és az extra neveket (a SAN-ban, helyettesítő névvel is);
- a tanúsítvány érvényes-e már, és nem járt-e le; 30 napon belüli lejáratnál figyelmeztet;
- a kulcs típusát és méretét (RSA legalább 2048 bit);
- van-e köztes tanúsítvány a fájlban.

Hibánál a web nem indul, és a napló megmondja az okot. Kézi próba a futó rendszeren:

```
docker compose exec web openhdid tls-check
```

A parancs az `OPENHDID_TLS=on` módhoz tartozik: `selfsigned` és `off` módban nincs mit ellenőriznie, a hiányzó tanúsítványfájl miatt hibával lép ki.

A beállítás kizárólag TLS 1.3-at fogad (`TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_AES_128_GCM_SHA256`). Early data és session ticket nincs. A kulcscsere elsődlegesen az X25519MLKEM768 poszt-kvantum hibrid eljárás, ezt a friss böngészők és az OpenSSL 3.5 támogatja, a régebbi kliensek az X25519-re esnek vissza. Minden HTTPS-válasz HSTS-fejlécet kap (1 év, aldomainekkel). A TLS 1.2 engedélyezése csak dokumentált döntéssel és klienspróbával jöhet szóba, régi partner miatt sem kerülhet vissza a TLS 1.0/1.1.

Tanúsítványcsere: a fájlok cseréje után kapcsolatbontás nélküli újratöltés, amely előtte ellenőriz:

```
docker compose exec web openhdid reload
```

Automatikus megújításnál (például ACME DNS-01 a gazdagépen) ezt a parancsot kösse a megújítás utáni lépéshez. A konténer nem nyit ACME HTTP-01 útvonalat.

6.4 Fejlécek és a kiszolgáló azonosítóinak elrejtése

Az nginx forrásból, csak a szükséges modulokkal készül; a headers-more modul be van fordítva.

- **Server és X-Powered-By:** egyik válaszban sincs, a hibaválaszokban sem (`server_tokens off`, `more_clear_headers`, `expose_php=Off`, `fastcgi_hide_header`).
- **Biztonsági fejlécek minden válaszon egyszer:** HSTS, `nosniff`, `X-Frame-Options DENY`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Opener-Policy`, CSP és `X-Robots-Tag`.
- **Gyorsítótár:** a dinamikus válaszokon `Cache-Control: no-store`; a tartalomhash-elt statikus fájlok egy évig gyorsítótárazhatók, előtömörítve.
- **Hibák és tiltások:** saját, termék- és verziómentes hibaoldal; csak az `index.php` futtatható, a pontos nevű fájlok nem érhetők el, a TRACE 405.
- **Access log:** nem tárol URL-t, lekérdezést, Referert, sütit és kéréstörzset.

6.5 Terheléselosztó vagy fordított proxy mögött

`OPENHDID_TLS=off` esetén a konténer csak a 8080-as porton, HTTP-n szolgál ki. A TLS-t ekkor a terheléselosztó zárja le, és ott is TLS 1.3 legyen.

- **`TRUSTED_PROXIES`:** a terheléselosztó címei vagy CIDR-tartománya, csak azok; a `*` nem használható. Az nginx csak ezektől fogadja el az `X-Forwarded-For` kliens-IP-t (realip), a PHP felé pedig minden módban a saját értékére cseréli az `X-Forwarded-Host`, `-Proto`, `-Port`, `-Prefix` és `Forwarded` fejlécet (az illesztett kiszolgálónév, https, az `APP_URL` portja), az `X-Forwarded-For` helyére a realip utáni címet teszi; így a megbízható tartomány egy másik gépe sem írathat idegen címet a belépési levélbe. A kéréskorlát a valódi kliens-címre kulcsol.
- **A 8080-as port** csak a terheléselosztó felől legyen elérhető (tűzfal vagy belső hálózat).
- **Élő-próba:** `GET /up` a `Host` fejlécben az `APP_URL` gépnevével, a terheléselosztó címével az `OPENHDID_MONITORING_ALLOW` listában (különben 404); a próba csak a PHP indulását mutatja.
- **Kulcsok:** a tanúsítvány és a kulcs a terheléselosztón él. Ha a belső szakaszt is titkosítani kell, maradjon `OPENHDID_TLS=on` a konténerben is (újratitkosítás vagy TLS-átengedés); ekkor az nginx a proxy címét látja, ezért a kliens-címenkénti kéréskorlát minden mögötte álló klienst egynek számol (a konténer induláskor figyelmeztet): emelje a korlátot, állítsa `0`-ra, vagy zárja le a TLS-t az előtéten (`off`).

HAProxy-minta (a telepítő a kétgépes 10k A és B gépén ugyanezt írja ki a saját címekkel; saját terheléselosztó esetén az használandó, ugyanezekkel a beállításokkal: TLS 1.3, HTTP → HTTPS 308, a kliens `X-Forwarded-For` fejlécét eldobja és a sajátját adja, `X-Forwarded-Proto: https`, `/up` élő-próba a `Host` fejléccel, a kiszolgálói időkorlát a PHP-FPM 330 mp-es `request_terminate_timeout`-jához igazítva):

```
frontend openhdid
    bind :443 ssl crt /etc/haproxy/certs/hdid.example.org.pem alpn h2,http/1.1 ssl-min-ver TLSv1.3
    bind :80
    http-request redirect scheme https code 308 unless { ssl_fc }
    http-request del-header X-Forwarded-For
    http-request set-header X-Forwarded-Proto https
    option forwardfor
    timeout client 60s
    default_backend openhdid_app
backend openhdid_app
    balance roundrobin
    timeout connect 5s
    timeout server 330s
    option httpchk
    http-check send meth GET uri /up ver HTTP/1.1 hdr Host hdid.example.org
    http-check expect status 200
    server a 10.0.0.11:80 check inter 5s fall 3 rise 2
    server b 10.0.0.12:80 check inter 5s fall 3 rise 2
```

A `.pem` a teljes lánc és a kulcs egy fájlban. Az elosztó címe a `TRUSTED_PROXIES` és az `OPENHDID_MONITORING_ALLOW` sorba kerül mindkét alkalmazásgépen (a telepítő ezt a terheléselosztó címéből maga írja), különben a kliens-cím az elosztóé lesz, és a `/up` 404-et ad.

6.6 Felügyeleti végpontok

A `/health` (az összes ellenőrzés állapota, 200 vagy 503) és a `/up` csak a konténeren belülről (127.0.0.1, a kapcsolat saját címe szerint: egy továbbított `X-Forwarded-For: 127.0.0.1` nem nyitja meg) és az `OPENHDID_MONITORING_ALLOW` címeiről érhető el, mások 404-et kapnak; terheléselosztó mögül a felügyeleti rendszert a továbbított címe szerint kell a listában felsorolni. A gazdagépről a közzétett porton érkező kérés a Docker-híd átjárócíméről jön (a `docker network inspect openhdid_default` mutatja, például 172.18.0.1), ezért ezt a címet kell a listába venni, különben a gazdagépről is 404 a válasz; a `docker compose exec web php artisan hdid:health` a listától függetlenül működik. A konténer saját egészség-ellenőrzése egy csak a konténeren belül elérhető porton fut, és nem függ ettől a listától.

6.7 Belső nevek feloldása a konténerből

A konténer a Docker beépített DNS-én át a gazdagép DNS-kiszolgálóit kérdezi, ezért a belső DNS-ben szereplő nevet (Exchange, ADFS, ügyféltörzs, SMS-átjáró, külső adatbázis) magától feloldja; nem kell minden nevet felvenni. A gazdagép `/etc/hosts` bejegyzéseit viszont nem látja.

- **A név csak a gazdagép `/etc/hosts`-ában van** (teszt gépen gyakori): `extra_hosts` egy, a `compose.yaml` mellé tett `compose.override.yaml`-ban a `web`, `worker`, `sync-worker` és `scheduler` szolgáltatásnak, majd `docker compose config --quiet && docker compose up -d`. Mivel a `.env` `COMPOSE_FILE` sora a használati profil miatt be van állítva, a Compose a `compose.override.yaml`-t magától nem olvassa: a fájlt a sor végére kell írni (`COMPOSE_FILE=compose.yaml:profiles/5k/compose.yaml:compose.override.yaml`, a 10k profilnál `profiles/10k/compose.yaml`). A kiadott `compose.yaml` érintetlen marad. Tartósan jobb, ha a név a belső DNS-be kerül.
- **A gazdagép helyi feloldót használ** (a `/etc/resolv.conf`-ban csak `127.0.0.x` áll, például systemd-resolved), vagy a belső DNS csak VPN-en át érhető el: a Dockernek a `/etc/docker/daemon.json` `dns` kulcsában egyszer kell megadni a belső DNS-kiszolgálókat (meglévő fájlnál abba felvéve), majd `sudo systemctl restart docker` és `docker compose up -d --force-recreate`. Így egyetlen nevet sem kell egyenként felvenni.

Egy név vizsgálata (gazdagép kontra konténer), a `compose.override.yaml`-minta és a `daemon.json`-parancsok a kiadás `TROUBLESHOOTING.md` mellékletében.

6.8 Kéréskorlát és túlterhelés

A webkonténer nginx-e az alkalmazás felé menő kéréseket (a statikus fájlokat nem) kliens-címenként korlátozza; a korlátok kulcsa IPv4-en a cím, IPv6-on a /64-es hálózat (egy előfizető egész /64-et kap, azon belül cserélhetné a címét). Az alkalmazás saját, címenkénti korlátai (a belépési útvonalak, a panel belépési korlátja, a gépi API címplafonja) ugyanígy számolnak (2026. október 26.).

- **Kérések:** legfeljebb 20 kérés/s és egyszerre 100-zal több (`OPENHDID_NGINX_RATE_LIMIT` , `OPENHDID_NGINX_RATE_BURST` ; 2026. október 26-ig 50 és 200, ami az 5k profil teljes normál csúcsával volt egyenlő), legfeljebb 16 egyidejű kérés (`OPENHDID_NGINX_CONN_LIMIT`); felette 429 a válasz, két nyelven az okkal és `Retry-After: 5` fejléccel. A `HDID_RATE_LIMIT_EXEMPT_NETWORKS` hálózataira a korlátok nem hatnak. Az indulási napló kiírja a korlátot („nginx: at most 20 requests/s per client address …”), `0` értéknél figyelmeztet. A naplóban: `docker compose logs web | grep 'limiting requests'` .
- **Túlterhelés:** ha nincs szabad PHP-FPM folyamat, és a várakozó sor (`OPENHDID_FPM_BACKLOG` , 64) is tele, az nginx azonnal 503-at ad két nyelven az okkal és `Retry-After: 10` fejléccel, ahelyett, hogy a kérés percekig várna; a megszakított kérés folyamata legfeljebb 125 mp után felszabadul.
- **Kliens és törzs:** a fejlécre 10 mp, a törzsre, a küldésre és a keepalive-ra 15 mp jut (`OPENHDID_NGINX_CLIENT_TIMEOUT`); a kéréstörzs legfeljebb 4 MB, a fájlfeltöltésé (`/livewire-…/upload-file`) 11 MB; HTTP/2-n legfeljebb 32 egyidejű stream.
- **Egy címről érkező forgalom:** ha minden kérés ugyanarról a címről érkezik (rootless Docker alapértelmezett port-továbbítója, IPv6-NAT nélküli IPv6-kliensek, újratitkosító előtét `OPENHDID_TLS=on` mellett, terheléselosztó `TRUSTED_PROXIES` nélkül), a korlát az egész forgalomra hat. NAT mögötti vállalati hálózatnál a `HDID_RATE_LIMIT_EXEMPT_NETWORKS` a javasolt megoldás; a többi esetben a valódi kliens-cím helyreállítása (2. és 3. fejezet, 6.5 pont), végső esetben a korlát emelése vagy `0` .
- **A méretezés elve:** a profilok a normál csúcs kétszeresére szólnak (15. fejezet); ami a korlátokon felül érkezik, azt a rendszer 429-cel vagy 503-mal elutasítja, nem omlik össze. A hálózati szintű, térfogati (L3/L4) DDoS nem az alkalmazás feladata: a szolgáltató DDoS-szűrése vagy egy előtét (terheléselosztó, WAF) védi ki. Ajánlott ECDSA-tanúsítvány (olcsóbb TLS-kézfogás) és a gazdagép tűzfalán új kapcsolat/s korlát.

---

7 Adatbázis (MariaDB)

7.1 Beépített adatbázis

Indítás és kötet. A `COMPOSE_PROFILES=db` a `mariadb:12.3` LTS image-et indítja digesttel rögzítve, a `db` nevű kötettel. A port a gazdagépre nincs kivezetve.

Első induláskor a MariaDB létrehozza:

- a `DB_DATABASE` adatbázist `utf8mb4_unicode_ci` rendezéssel;
- a migrációs fiókot (a `db.env` `DB_USERNAME` / `DB_PASSWORD` értéke), minden joggal az adatbázison;
- a futási fiókot (a `.env` `DB_USERNAME` / `DB_PASSWORD` értéke), csak `SELECT, INSERT, UPDATE, DELETE` joggal.

Ki melyik fiókot használja. A web, a worker és a scheduler csak a futási fiókot látja, sémát nem módosíthat. A migrációk a migrációs fiókkal futnak.

Kiszolgálóbeállítás. Karakterkészlet, `skip-name-resolve`, puffer (`OPENHDID_DB_BUFFER_POOL`), kapcsolatszám (`OPENHDID_DB_MAX_CONNECTIONS`) és memóriaplafon (`OPENHDID_DB_MEM_LIMIT`) a használati profil szerint (5. és 15. fejezet).

> **Csak az első induláskor.**
> A fiókokat és az adatbázist a MariaDB az üres kötet első indításakor hozza létre. Jelszócseréhez később a `mariadb` konténerben `ALTER USER` kell, és a `.env` / `db.env` ezzel egyezzen. Fájlból adott titoknál (`*_FILE`) a fájl tartalma számít, ezért jelszócserénél a fájlt és az adatbázis-fiókot együtt kell módosítani.

7.2 Külső adatbázis

A `COMPOSE_PROFILES` üres (vagy csak `redis`). A `DB_HOST` a kiszolgáló neve. Az adatbázist és a két fiókot előre kell létrehozni:

```
CREATE DATABASE openhdid CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'openhdid'@'10.0.0.%' IDENTIFIED BY '…futási jelszó…';
GRANT SELECT, INSERT, UPDATE, DELETE ON openhdid.* TO 'openhdid'@'10.0.0.%';
CREATE USER 'openhdid_migrate'@'10.0.0.%' IDENTIFIED BY '…migrációs jelszó…';
GRANT ALL PRIVILEGES ON openhdid.* TO 'openhdid_migrate'@'10.0.0.%';
```

Infra gép a telepítővel (kétgépes 10k, 15.8 pont): ugyanezt a két fiókot a `profiles/10k/infra.yaml` MariaDB-je az első induláskor maga hozza létre (a futási fiók csak `SELECT, INSERT, UPDATE, DELETE`, a migrációs fiók `ALL` jogot kap az adatbázisra, mindkettő `'%'` gépről, ezért a 3306-os port csak az A és a B gép felől legyen elérhető); a jelszavakat a telepítő készíti, a futásit a gép `.env`-jébe, a migrációsat és a rendszergazdáét a `db.env`-be írja. Az `install.sh check` `INFRA-04` lelete a két fiók jogait ellenőrzi.

A hálózaton át a kapcsolat legyen TLS-es: kényszerítse a kiszolgálón (`REQUIRE SSL` vagy `require_secure_transport`), a CA-fájlt tegye a `ca` mappába, és adja meg a `MYSQL_ATTR_SSL_CA=/etc/openhdid/ca/<fájl>` értéket. A `db.env` ekkor is kell a migrációs fiókhoz; a `MARIADB_ROOT_PASSWORD` üresen maradhat.

7.3 Migrációk

- **Mikor fut:** a `migrate` szolgáltatás minden `docker compose up` alkalmával lefut. Ha nincs új migráció, nem változtat semmin.
- **Mit csinál:** legfeljebb 120 másodpercig vár az adatbázisra (`OPENHDID_DB_WAIT_SECONDS`), futtatja a migrációkat, felveszi a hiányzó jogosultságokat és beépített szerepköröket (a kézi módosításokat megőrzi), és a `db.env` migrációs fiókjával dolgozik.
- **Sikertelen migráció:** ha a `migrate` nem 0-val lép ki, a web és a háttérfolyamatok nem indulnak (`docker compose logs migrate`).
- **Csak előre:** a migrációk csak előre futnak; visszalépéshez a frissítés előtti mentés kell.
- **Egyszerre egy:** egyszerre csak egy `migrate` fusson. Valkey mellett a `migrate` a közös gyorsítótár zárja alatt fut (`migrate --isolated`): a 10k profil két alkalmazásgépe egyszerre is indulhat, a második megvárja az elsőt. Kubernetesben egyetlen Job, nem a webpodok indulása.

8 Háttérfolyamatok és ütemezés (a cron helyett)

A konténerekben nincs cron-démon, és a gazdagépen sem kell crontab-bejegyzés. Az ütemezett feladatokat a `scheduler` szolgáltatás futtatja (`php artisan schedule:work`): percenként végrehajtja az alkalmazás ütemezését.

Aktiválás: a feladatok maguktól élnek, amíg a `scheduler` fut; ez az alap Compose-telepítés része. Környezeti változóból csak két feladat állítható, a többi időzítése rögzített:

- `EMD_SYNC_CRON` : a teljes átvétel cron-kifejezése; üresen csak kézi átvétel van;
- `EMD_SYNC_TOKEN_KEEP_ALIVE` : a tokenfrissítés be- és kikapcsolása.

Az időpontok az `APP_TIMEZONE` szerint értendők.

GYAKORISÁG	FELADAT
percenként	lejárt azonosítási munkamenetek zárása; ütemező- és sor-életjel; elakadt kimenő üzenetek újrasorolása
15 percenként	`hdid:emd-sync-token` az ügyféltörzs API-tokenjének frissentartása (fájlos illesztő és bekapcsolt `EMD_SYNC_TOKEN_KEEP_ALIVE` mellett)
óránként	lejárt csomag-felülírások zárása; befejezett hívások tisztítása; végesemény nélkül maradt hívások lezárása; a főügyfél-jogosultságukat vesztett ügyfelek kapcsolatainak megszüntetése (védőháló, a csomagváltozás mentésekor azonnal lefut)
naponta 03:30	`hdid:prune` adatmegőrzési tisztítás és az auditnapló archiválása (a beállított kereten túli napok tömörített fájlokba a `storage` kötetre; üzemeltetési kézikönyv 11. fejezet)
`EMD_SYNC_CRON` szerint	`hdid:emd-sync` teljes ügyféltörzs-átvétel (alapból 3 óránként)

Ellenőrzés: `docker compose exec scheduler php artisan schedule:list` , valamint a *Rendszerállapot* oldal „Ütemező” és „Feladatsor” kártyája, amely a percenkénti életjelből dolgozik.

Az ütemezőből mindig pontosan egy példány fusson. A kritikus feladatok a közös gyorsítótár záraival védettek (`withoutOverlapping` , `onOneServer`), így egy véletlen második példány sem futtatja őket kétszer, de ettől még nem kívánatos.

A `worker` és a `sync-worker` :

- óránként, illetve 192 MB fölött a feladat után szabályosan kilépnek (`--max-time=3600` , `--memory=192`), és a Compose újraindítja őket; a konténer plafonja 384 MB (`sync-worker` : 448 MB), így a memória nem nő korlátlanul;
- minden feladat előtt újraolvassák a felületi beállításokat;
- a panelről indított ügyféltörzs-átvétel csak akkor fut, ha a `sync-worker` fut, és az `EMD_SYNC_QUEUE` értéke `sync` ; különben a rendszerállapot 5 perc várakozás után figyelmeztet.

9 Naplók

Minden folyamat a konténer szabványos kimenetére ír, fájlba nem. A konténermotor gyűjti és forgatja a naplókat: a `compose.yaml` `json-file` naplózó-meghajtót használ, szolgáltatásonként legfeljebb 10 × 20 MB-tal (Podman alatt ez a `k8s-file`, ott csak a méretkorlát él). Beállítás nélkül a naplók korlátlanul nőnének; a Docker saját `local` meghajtóját a Podman nem ismeri.

FORRÁS	FORMÁTUM	OLVASÁS
nginx access log	soronként JSON: idő, kliens-IP, kérésazonosító, metódus, státusz, méret, időtartam	`docker compose logs -f web`
nginx hibák, PHP-FPM, alkalmazás	szöveges sorok; az alkalmazás JSON-ra is állítható (`LOG_STDERR_FORMATTER=Monolog\Formatter\JsonFormatter`)	`docker compose logs web`
üzenetsor-feldolgozók	szöveges sorok	`docker compose logs -f worker sync-worker`
ütemező	szöveges sorok	`docker compose logs scheduler`
migráció, adatbázis	szöveges sorok	`docker compose logs migrate mariadb`
auditnapló	adatbázis	felület: *Rendszer › Auditnapló*

Hasznos szűrések:

```
docker compose logs --since 1h web                    # az utolsó óra
docker compose logs --no-log-prefix web | grep '"status":5'   # szerverhibák
docker compose logs --no-log-prefix web | grep -v '^{' # minden, ami nem access log (hibák, figyelmeztetések)
docker compose logs -f --tail=100 worker             # élő követés
docker compose logs --no-log-prefix web | grep 'limiting requests'   # kéréskorlát (429)
```

Központi naplógyűjtés. Az ASVS V16 elkülönített, módosítás ellen védett tárolást vár. A `compose.yaml` `x-logging` blokkjában a meghajtó cserélhető, például `journald`, `syslog` (TLS-sel), `gelf` vagy `fluentd`; a Docker-démon szintjén is beállítható. Az access log nem tartalmaz URL-t és titkot. Az nginx hibasorai és az alkalmazás hibaszövegei viszont tartalmazhatnak URL-t vagy személyes adatot, ezért a továbbításuk korlátozott hozzáféréssel történjen.

Fájlnapló (ha a szervezet ezt kéri): `LOG_STACK=stderr,daily` és `LOG_CHANNEL=stack`. A napi fájlok ekkor a `storage` köteten keletkeznek (`storage/logs`), 30 napos megőrzéssel.

10 Első beállítás a felületen

Belépés a `/admin` címen az első rendszergazdával, majd az *Adminisztráció › Rendszer › Beállítások* alatt, ebben a sorrendben:

1. **Csomagok:** a prémium (és ha van, a normál) csomagnevek listája. Csomaglista nélkül egyetlen ügyfél sem tud belépni.
2. **Ügyfélszolgálat:** szintenként e-mail-cím, telefonszám, nyitvatartás.
3. **Arculat és Prémium:** alkalmazásnév, embléma, böngészőikon, színek, háttérképek. Az alapértelmezés semleges színséma; a levelek és a kimutatások is a beállított színeket használják.
4. **Portál:** jogi név, lábléc, adatkezelési és felhasználási hivatkozások.
5. **Ügyfél belépés és SSO:** mely belépési módok élnek. Az SSO bekapcsolása előtt a környezeti változók legyenek kitöltve; több Entra-bérlőnél a „Céges belépés: Microsoft Entra ID” kapcsoló alatt ellenőrizze a bérlők domainjeinek állapotát. A névtelenül kért belépési üzenetek rendszerszintű plafonja (Ügyfélportál › Belépési módok): belépési hivatkozás óránként legfeljebb 10 000 (0 = nincs plafon), SMS-belépőkód óránként 60 és naponta 300, egy számra naponta 5; a hivatkozás plafonját a levelezőszolgáltató valódi korlátjához igazítsa (például egy Exchange Online-postafiókról percenként 30 levél, 24 óra alatt 10 000 címzett; nagy telepítésnél több küldő postafiók vagy levéltovábbító). Betelt plafonnál a portál belépő oldala jelzi, hogy a hivatkozás vagy a kód most nem kérhető.
6. **Ügyféltörzs (EMD):** munkatársi és ügyfél-domainlista, oszlopmegfeleltetés, védőkorlátok, CSV-kódolás, az export és az ID-lista kéréstörzse, az ID-lista válaszútvonalai és kijelölése (az ügyféltörzs-átvétel kezelésére jogosult munkatárs).
7. **Azonosítás, Hívások, Adatmegőrzés:** az alapértékek az induláshoz megfelelnek. Az ügyintézői sikertelen ellenőrzések korlátja alapból 50 óránként. Új: ügyfelenként legfeljebb 3 sikertelen kérdés-válasz munkamenet 24 órán belül, és az „Elfogadott országok (telefonszám)” (alapból Magyarország): a portál csak ezek számát veszi fel, SMS csak ezek mobilszámára megy.

Ezután: *Tartalom › Üzenetsablonok* (próbaküldés), *Rendszer › API-kulcsok*, *Fiókok* (munkatársak, szerepkörök), *Azonosítás › Kérdések*. A felület részletes leírása a felhasználói kézikönyvben található.

---

11 Külső rendszerek bekötése

A konténerben futó parancsok előtagja `docker compose exec web php artisan`.

Ügyféltörzs (Enterprise Master Data, EMD)

1. Töltse ki az `EMD_SYNC_*` értékeket (jelentésük az 5. fejezet táblázatában), majd `docker compose up -d`. Az export és az ID-lista kéréstörzse nem környezeti változó: a panel *Beállítások › Ügyféltörzs (EMD)* fülén áll.
2. A belépés ellenőrzése: `docker compose exec web php artisan hdid:emd-sync-token`.
3. Próbafuttatás: `docker compose exec web php artisan hdid:emd-sync --dry-run`, vagy a panel *Próbafuttatás az API-ból* gombja; utóbbi a `sync-worker`-en fut.
4. Megfelelő eredmény után az éles átvétel.

A védőkorlátok, a domainszűrés és a hibakezelés az üzemeltetési kézikönyv 6. fejezetében olvashatók.

Levelezés

`MAIL_MAILER=ews` az `EWS_*` értékekkel, vagy `smtp` a `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD` és `MAIL_SCHEME` értékekkel; SMTP-n a STARTTLS kötelező (`MAIL_REQUIRE_TLS`), STARTTLS nélküli kiszolgálónál `MAIL_SCHEME=smtps` a 465-ös porttal. A feladó a `MAIL_FROM_ADDRESS`.

EWS-en a levelek mentett másolat nélkül mennek; az Exchange saját naplózását, archiválását vagy átviteli szabályát az Exchange gazdája szabályozza. Próba az ügyfelek érintése nélkül:

```
docker compose exec web php artisan hdid:test-mail sajat@example.org --now
```

SMS (Ozeki NG)

`SMS_DRIVER=ozeki`, az átjáró HTTPS-címe, felhasználó és jelszó. Próba: `docker compose exec web php artisan hdid:test-sms +36301234567 --now`. Az átjáró felé POST-kérés megy, a hozzáférés nem kerül a címsorba.

Kézbesítési jelentés: az „elküldve” állapot az átjáró elfogadása; a rendszer az átjáró üzenetazonosítójával az `SMS_DELIVERY_CHECKS` időpontjaiban (alapból 30 mp, 2, 5, 30 és 120 perc után, legfeljebb ötször) kérdezi le az állapotot, és az üzenet „kézbesítve” vagy „nem kézbesíthető” lesz, jelentés nélkül „nincs jelentés”. A lekérdező művelet (`OZEKI_STATUS_ACTION`, alapból `querymessage`) és az állapotszavak (`OZEKI_DELIVERED_STATUSES`, `OZEKI_UNDELIVERED_STATUSES`) az Ozeki verziójától függnek: próba-SMS után az üzenet adatlapja (Kimenő üzenetek) mutatja az átjáró nyers állapotszavát, „a lekérdezés nem sikerült” esetén a műveletet az átjáró HTTP API-dokumentációja szerint kell megadni. A nem kézbesíthető SMS-t a rendszer nem küldi újra magától.

Egyszeri bejelentkezés (Entra ID, ADFS)

Négy alkalmazás-regisztráció kell: Entra és ADFS, munkatársaknak és ügyfeleknek külön. A mobilalkalmazás saját nyilvános ügyfélazonosítót kap.

Visszatérési cím: `https://<kiszolgáló>/auth/{user|client}/{entra|adfs}/callback`.

A fiók kötése a címtár-azonosítóhoz:

- az e-mail-cím (vagy a dolgozói azonosító) csak megtalálja a fiókot; az első jogosult céges belépés a fiókot a szolgáltató állandó azonosítójához köti (Entra `oid` a bérlővel, ADFS `sub` a regisztrációval), utána ugyanarra a címre más azonosítóval érkező belépés „más címtár-azonosító” okkal elutasul, a fiók nem zárolódik;
- frissítés után minden meglévő fiók az első belépésekor kötődik; a feloldás rendszergazdai döntés: a lista vagy az adatlap `Belépési kötés törlése` gombja, vagy `docker compose exec web php artisan hdid:sso-unbind email@example.org`;
- az ügyféltörzs szabálya változatlan: a tétel e-mail-címének a fiókéval egyeznie kell; az Entra objektumazonosítója a böngészős és a mobil regisztrációnál ugyanaz;
- ADFS újraregisztrálása (új kliensazonosító) után a következő jogosult belépés a kötést magától megújítja; a titok cseréje nem érinti;
- `HDID_SSO_REQUIRE_SUBJECT=false` csak olyan szolgáltatónál, amely az azonosítót nem adja ki; a Rendszerállapot „Belépési kötések” kártyája a más azonosító miatti elutasításokat mutatja;
- kilépéskor az alkalmazás csak a saját munkamenetét és tokenjét szünteti meg, a Microsoft- vagy ADFS-munkamenetet nem: közös gépen a céges belépés gombja jelszó nélkül visszaléptethet, a védelem a munkaállomás zárolása.

Dolgozói azonosító mint a fiókot megtaláló igény (`USER_SSO_MATCH=employee_id`, ügyfeleknél `CLIENT_SSO_MATCH`): a token dolgozói azonosítója találja meg az ügyféltörzstételt (külső azonosító) és azon át a fiókot; a tétel és a fiók e-mail-egyezése marad, a token e-mail-címe nem számít. Az igény neve a `USER_SSO_EMPLOYEE_ID_CLAIM` (üresen `employeeId`). Kiadatása Entránál igény-leképezési szabályzattal (`user.employeeid` → `employeeId`, a manifestben `acceptMappedClaims: true`), ADFS-nél a Web API kiadási szabályával (`employeeID` → `employeeId`). A helyben felvett tétel (`hdid:make-admin`) ebben a módban nem léphet be; hiányzó igénynél az auditnapló az `employee_id_claim` és `employee_id_problem` mezőben mutatja az okot.

Entra-bérlő:

- a saját címtár GUID-ja kell (domain, `common` vagy `organizations` nem jó);
- a regisztráció legyen egybérlős;
- az `email` igény csak ellenőrzött állapotban számít, különben a felhasználónév (UPN) dönt.

Ügyfélportál több Entra-bérlőből:

- `CLIENT_ENTRA_TENANTS`: a bérlők GUID-jai vesszővel, például `1111…,2222…:partner.hu|partner.com`; a kettőspont utáni domainek szűrők (a bérlő csak ezek fiókjait léptetheti be), domain nélkül a bérlő megbízható;
- több bérlőnél a rendszer a közös `organizations` végpontot használja, a token bérlőjét a listához, a kibocsátót a token saját bérlőjéhez köti; a listán kívüli bérlő tokenje érvénytelen;
- az ügyfél-alkalmazás (és a mobilalkalmazás) regisztrációja legyen többbérlős; a partnerbérlő rendszergazdája az első belépéskor hozzájárul; vendégfelhasználó a saját bérlője azonosítójával jön, annak is a listán kell lennie;
- a fiók kötése bérlőnként él: másik engedélyezett bérlőből érkező azonos cím „más címtár-azonosító”, csak a listáról lekerült bérlő kötése újul meg;
- a `CLIENT_ENTRA_TENANT` és a `CLIENT_ENTRA_TENANTS` együtt is működik, de a Rendszerállapot sárga, amíg az egyiket ki nem veszi; `common`, `organizations`, nem GUID alakú azonosító vagy hibás domain mellett „nincs beállítva”, a kártya megnevezi az okot; ugyanaz a domain több bérlőnél sárga jelzés. A munkatársi belépés egy bérlős marad.

Entra beállítása lépésenként, egy bérlő. Ha az alkalmazás rendszergazdája egyben az Entra rendszergazdája is, ez a menet elég. Az ügyfélportál regisztrációját írja le; a munkatársi panel regisztrációja ugyanez a `USER_ENTRA_*` kulcsokkal és a `/auth/user/entra/callback` visszatérési címmel, mindig egy bérlővel.

1. **Regisztráció.** Entra admin központ › *App registrations* › *New registration*. Név például „HDID ügyfélportál”; *Supported account types*: „Accounts in this organizational directory only” (egybérlős); *Redirect URI*: platform *Web*, cím `https://hdid.example.org/auth/client/entra/callback`.
2. **Azonosítók.** Az *Overview* lapon az *Application (client) ID* a `CLIENT_ENTRA_CLIENT_ID`, a *Directory (tenant) ID* a `CLIENT_ENTRA_TENANT` (GUID; a rendszer csak ezt az alakot fogadja el).
3. **Titok.** *Certificates & secrets* › *New client secret*; az érték csak létrehozáskor látszik, ez a `CLIENT_ENTRA_CLIENT_SECRET`. Jegyezze fel a lejáratát, és lejárat előtt cserélje (új titok, `.env`, újraindítás).
4. **Igények.** *Token configuration* › *Add optional claim* › *ID*: `email` és `xms_edov`. Az `xms_edov` nélkül a rendszer az `email` igényt nem fogadja el fiókazonosításra, és a felhasználónév (UPN) dönt; ha a UPN és a levelezési cím eltér, a belépés „nincs fiók” hibát ad.
5. **Engedélyek.** *API permissions*: a `User.Read` alapértelmezett jogosultság elég (az `openid`, `profile`, `email` hatókört a rendszer kéri); *Grant admin consent for <bérlő>*, így a felhasználók nem kapnak hozzájárulási kérdést.

6. **Ki léphet be.** *Enterprise applications* › az alkalmazás › *Properties* › *Assignment required: Yes*, majd *Users and groups* alatt a jogosult csoport; így a Microsoft is csak a kijelölteket engedi az alkalmazáshoz. A rendszer ezen felül az ügyféltörzs szerint dönt.
7. **Mobilalkalmazás.** Külön, nyilvános kliensként regisztrálva (platform *Mobile and desktop applications*, az MSAL visszatérési címével), ugyanebben a bérlőben; az *Application (client) ID* a `CLIENT_ENTRA_MOBILE_CLIENT_ID`.
8. **HDID.** A `.env` kulcsai, utána `docker compose up -d`; a *Beállítások › Ügyfélportál* fülön a „Céges belépés: Microsoft Entra ID” bekapcsolása (a mező alatti sor mutatja, megvan-e minden kulcs).

   ```
   CLIENT_ENTRA_TENANT=72f988bf-86f1-41af-91ab-2d7cd011db47
   CLIENT_ENTRA_CLIENT_ID=<Application (client) ID>
   CLIENT_ENTRA_CLIENT_SECRET=<a titok értéke>
   CLIENT_ENTRA_MOBILE_CLIENT_ID=<a mobil regisztráció client ID-ja>
   ```

9. **Ellenőrzés.** A Rendszerállapot „Belépési szolgáltatók” kártyája zöld és „1 bérlő” áll rajta; egy próbabelépés után az auditnapló `login.succeeded` bejegyzésében `email_claim: email` (vagy `preferred_username`), `tenant` a bérlő azonosítója, `sso_subject: bound`.

Entra beállítása lépésenként, több bérlő. Akkor kell, ha az ügyfelek más szervezetek címtáraiból lépnek be. Az előző menetre épül; a különbségek:

1. **Többbérlős regisztráció.** Az ügyfélportál (és a mobil) regisztrációjának *Authentication* › *Supported account types* beállítása „Accounts in any organizational directory (Any Microsoft Entra ID tenant – Multitenant)”. A visszatérési cím és a titok nem változik. A magánfiókokat is engedő változatot („…and personal Microsoft accounts”) ne válassza: a rendszer csak szervezeti bérlőket fogad.
2. **A partner bérlőazonosítója.** A partner rendszergazdája az Entra admin központ *Overview* lapján olvassa ki (*Tenant ID*); ellenőrzésképp a `https://login.microsoftonline.com/<partner domainje>/.well-known/openid-configuration` címen a `issuer` mezőben ugyanez a GUID áll.
3. **HDID.** A `CLIENT_ENTRA_TENANTS` sorolja fel a bérlőket; a saját bérlő is kerüljön a listába, és a `CLIENT_ENTRA_TENANT` maradjon üres (ha mindkettő ki van töltve, a rendszer működik, de a Rendszerállapot sárgán kéri a tisztítást). A kettőspont utáni domainek szűrők, domain nélkül a bérlő megbízható.

   ```
   CLIENT_ENTRA_TENANT=
   CLIENT_ENTRA_TENANTS=72f988bf-86f1-41af-91ab-2d7cd011db47,9c2d1a7e-5b34-4f0e-8a6b-0f1e2d3c4b5a:partner.hu|partner.com
   CLIENT_ENTRA_CLIENT_ID=<Application (client) ID>
   CLIENT_ENTRA_CLIENT_SECRET=<a titok értéke>
   CLIENT_ENTRA_MOBILE_CLIENT_ID=<a mobil regisztráció client ID-ja>
   ```

   A példában a saját bérlő bármely ügyféltörzsbeli címet beléptethet, a partner bérlője csak a `partner.hu` és `partner.com` címűeket. Utána `docker compose up -d`.

4. **Hozzájárulás a partner oldalán.** A partner bérlőjében az első belépéskor a Microsoft hozzájárulást kér az alkalmazáshoz. A legegyszerűbb, ha a partner rendszergazdája lép be elsőként, és bejelöli a *Consent on behalf of your organization* lehetőséget; alternatíva az előzetes hozzájárulás a `https://login.microsoftonline.com/<partner GUID>/adminconsent?client_id=<CLIENT_ENTRA_CLIENT_ID>` címen (a Microsoft a hozzájárulás után a visszatérési címre irányít, ahol

a belépő oldal egy „érvénytelen hitelesítő adatok” üzenetet mutathat; ez csak a félbehagyott belépés jele, a hozzájárulás él). Mivel az alkalmazás nem igazolt kiadójú, a partner bérlőjének alapbeállítása mellett a felhasználói hozzájárulás nem elég, a rendszergazdai kell.

5. **Mobilalkalmazás.** Az MSAL *authority* értéke `https://login.microsoftonline.com/organizations` a bérlőazonosítós helyett.
6. **Ellenőrzés.** A „Belépési szolgáltatók” kártya a bérlők számát és a domain-szűrőket mutatja; piros, ha egy tétel nem GUID vagy közös végpontnév, sárga, ha a két bérlőkulcs együtt van kitöltve, vagy egy domain több bérlőnél szerepel. Egy próbabelépés minden bérlőből: az auditnapló `login.succeeded` bejegyzésének `tenant` mezője mutatja, melyik bérlőből jött; a listán kívüli bérlő `login.rejected`, `ID token tenant not allowed` hibával látszik, a domain-szűrőn fennakadt cím `tenant_domain_not_allowed` okkal.

ADFS, a fiókot kötő igény:

- a rendszer a `USER_ADFS_EMAIL_CLAIM` (ügyfeleknél `CLIENT_ADFS_EMAIL_CLAIM`) igényeit sorrendben nézi, az első e-mail-alakú érték dönt; üresen `emailaddress`, `email`, `upn`, `preferred_username`, `unique_name`;
- a név a teljes igénytípus-URI végére is illeszkedik (`…/claims/emailaddress`);
- minden igénynél előbb az ID token, utána az access token számít: a Web API kiadási szabályainak igényei `allatclaims` nélkül csak az access tokenbe kerülnek; az access token akkor számít, ha ugyanazzal a kulccsal aláírt, ettől az ADFS-től származik, nem járt le és ennek az alkalmazásnak szól (`appid`);
- a Web API azonosítója a `USER_ADFS_RESOURCE`, további hatókör a `USER_ADFS_SCOPES` (egyik sem kötelező);
- hogy egy belépés melyik igénnyel sikerült, vagy miért nem, az auditnapló mutatja (az igények nevével, értékük nélkül); a beállítás idejére a `HDID_ADFS_DEBUG=true` a tokenek tartalmát a naplóba írja. Módosítás után `docker compose up -d`.

ADFS beállítása lépésenként. Munkatársaknak és ügyfeleknek külön alkalmazáscsoport kell, ugyanezekkel a lépésekkel (ügyfeleknél a `CLIENT_ADFS_*` kulcsokkal és a `client` visszatérési címmel). Az első négy lépés az ADFS-kiszolgálón, az AD FS Management konzolban történik.

1. **Alkalmazáscsoport.** *Application Groups › Add Application Group*, sablon: „Server application accessing a web API”. Meglévő csoportnál: a csoport tulajdonságai › *Add application…* gomb.
2. **Szerveralkalmazás.** A *Client Identifier* lesz a `USER_ADFS_CLIENT_ID`. Visszatérési cím (*Redirect URI*): `https://hdid.example.org/auth/user/adfs/callback`. A *Configure Application Credentials* lépésben *Generate a shared secret*: ez a `USER_ADFS_CLIENT_SECRET`, és csak itt látszik.
3. **Web API.** Azonosító (*Identifier*): a HDID címe, például `https://hdid.example.org/`. Ugyanez kerül betűre azonosan a `USER_ADFS_RESOURCE` kulcsba (a záró perjel is számít). Az azonosító az ADFS-ben egyedi, ezért az ügyfelek csoportjának Web API-ja eltérőt kap, például `https://hdid.example.org/client/`. Hozzáférési szabályzat (*Access control policy*): a szervezet belépési szabályzata, például „Permit everyone” vagy többlépcsős azonosítást kérő szabályzat. Engedély (*Client Permissions*): a 2. lépés szerveralkalmazása, `openid`, `profile` és `email` hatókörrel (a HDID ezeket kéri).
4. **A cím kiadása.** A Web API *Issuance Transform Rules* fülén *Add Rule › Send LDAP Attributes as Claims*, attribútumtár: Active Directory, `E-Mail-Addresses` → `E-Mail Address`. Az ADFS ezt az igényt `allatclaims` hatókör nélkül csak az access tokenbe teszi, `email` néven.

5. **HDID.** A `.env` kulcsai, utána `docker compose up -d`; végül a *Beállítások › Munkatársi belépés* fülön a „Céges belépés: ADFS” bekapcsolása (a mező alatti sor mutatja, megvan-e minden kulcs).

   ```
   USER_ADFS_ISSUER=https://adfs.example.org/adfs
   USER_ADFS_CLIENT_ID=<Client Identifier>
   USER_ADFS_CLIENT_SECRET=<a megosztott titok>
   USER_ADFS_RESOURCE=https://hdid.example.org/
   USER_ADFS_EMAIL_CLAIM=email
   ```

   A `USER_ADFS_EMAIL_CLAIM=email` csak az e-mail-igénnyel köti a fiókot: ha az igény hiányzik, a belépés érthető üzenettel elutasul, és a rendszer nem áll át csendben a UPN-re (üresen a lista végén a `upn` is szerepel).
6. **Ellenőrzés.** Egy belépés után az auditnapló `login.succeeded` bejegyzésében `email_claim: email` és `email_token: access_token` áll, a rendszerállapot „Belépési szolgáltatók” kártyája a keresett igényt mutatja. Ha nem így van: az üzemeltetési kézikönyv „Az ADFS-belépés nem találja a fiókot” pontja; a leggyakoribb ok a hiányzó Web API vagy `USER_ADFS_RESOURCE`.

Ellenőrzés PowerShell-lel. Az `ADFS` PowerShell-modul csak az ADFS-kiszolgálókon van meg (RSAT-csomag nincs hozzá, a Web Application Proxyn sincs). Máshonnan távoli munkamenettel futtatható, az ADFS-kiszolgálón helyi rendszergazdai joggal (`adfs01` helyett az ADFS-kiszolgáló neve):

```
Invoke-Command -ComputerName adfs01 -ScriptBlock {
    Get-AdfsServerApplication | Format-List Name, Identifier, ApplicationGroupIdentifier
    Get-AdfsWebApiApplication | Format-List Name, Identifier, ApplicationGroupIdentifier
    Get-AdfsApplicationPermission | Format-List ClientRoleIdentifier,
ServerRoleIdentifier, ScopeNames
}
```

A szerveralkalmazás `Identifier` értéke a kliensazonosító. A HDID csoportjában kell lennie egy Web API-nak, amelynek `Identifier` listája tartalmazza a `USER_ADFS_RESOURCE` értékét, és egy engedélynek, amelyben a `ClientRoleIdentifier` a kliensazonosító, a `ServerRoleIdentifier` a Web API azonosítója.

Telefonközpont és automata menü

Kulcs. A panelen *Rendszer › API-kulcsok*, „Call center / IVR” hatókörrel; vagy parancssorból: `docker compose exec web php artisan hdid:api-key:create "Telefonközpont" callcenter`. A kulcs egyszer jelenik meg.

Küldés. A kulcs az `X-Api-Key` fejlécben küldendő. Az `api_key` lekérdezési paraméter csak örökölt partnernek használható, mert webszerver-naplóba kerülhet; az access log itt nem rögzíti.

Aláírás. Választható HMAC-aláírás: `X-Timestamp` és `X-Signature` fejléc. Az aláírandó szöveg: időbélyeg, metódus, kéréscím a lekérdezéssel, törzs.

Leírás. A teljes gépi felület rendszergazdával belépve a `/docs/ivr` címen olvasható, a mobil kiszolgálóé a `/docs/mobile` címen. A megjelenítőt a böngésző az `unpkg.com` címről tölti be (a tartalombiztonsági szabályzat ezt csak a `/docs/*` oldalakon engedi), ezért a munkaállomásnak internet-elérés kell; zárt hálózaton a `/docs/ivr.json` letölthető, és bármely OpenAPI-nézőben olvasható, ez adható át a telefonközpont szállítójának is.

12 Ellenőrzés és biztonsági átvétel

12.1 Működés

- ☐ `docker compose ps`: a `web` `healthy`, a `worker`, `sync-worker`, `scheduler` és `mariadb` fut, a `migrate` `exited (0)`.
- ☐ `docker compose exec web php artisan hdid:health` kilépési kódja 0; a Rendszerállapot oldalon az adatbázis, gyorsítótár, üzenetsor és ütemező zöld (az ütemező az első percben még piros lehet). Ha az ügyféltörzs API-belépése nincs beállítva (csak fájlimport), a „Belépés az ügyféltörzs API-jába” csempe piros marad, és a kód 2.
- ☐ A *Rendszerállapot* fejlécében a futó verzió látszik (a telepített kiadás száma, „Verzió: …”).
- ☐ Próbalevél és próba-SMS megérkezik; a *Kimenő üzenetek* listában „elküldve”.
- ☐ Az ügyféltörzs próbafuttatása hihető számokat ad, az éles átvétel sikeres.
- ☐ Egy ügyfélnek küldött belépési hivatkozás beléptet a portálra; bekapcsolt SSO-nál a belépés végigmegy.
- ☐ Telefonközponti próbahívás megjelenik a vezérlőpulton.
- ☐ A `docker compose logs web` access logjában a kliensek valódi IP-címe látszik (2. fejezet).

12.2 Biztonsági próbák a gazdagépről

```
H=hdid.example.org
curl -sS -D - -o /dev/null https://$H/login          # 200, Cache-Control: no-store
curl -sS -D - -o /dev/null https://$H/favicon.ico    # 200
curl -sS -D - -o /dev/null https://$H/nincs.css      # 404, saját hibaoldal
curl -sS -o /dev/null -w '%{http_code}\n' -X TRACE https://$H/   # 405
curl -sS -o /dev/null -w '%{http_code}\n' https://$H/.env       # 404
curl -sS -D - -o /dev/null http://$H/login           # 308 https://$H/login
openssl s_client -connect $H:443 -servername $H -tls1_3 -verify_return_error -brief
</dev/null   # siker
openssl s_client -connect $H:443 -servername $H -tls1_2 -brief </dev/null
# elutasítva
```

Elvárás: egyetlen válaszban sincs `Server` vagy `X-Powered-By` fejléc; a HSTS és a többi biztonsági fejléc pontosan egyszer szerepel; ismeretlen kiszolgálónévre a kézfogás elutasított. Belső CA esetén a CA-fájlt adja át (`--cacert`, `-CAfile`); a `-k` nem elfogadható bizonyíték.

12.3 OWASP-megfeleltetés

Alap: OWASP ASVS 5.0.0, Top 10:2025, OWASP Docker Security Cheat Sheet. A táblázat a konténeres telepítés védelmeit sorolja fel; teljes ASVS-megfelelőséget nem állít.

TERÜLET	VÉDELEM
A02 hibás konfiguráció, ASVS V13	Nem root (uid 82), csak olvasható gyökér-fájlrendszer, minden Linux-képesség elvéve, `no-new-privileges`, setuid-bitek nélkül. Nincs Server/X-Powered-By fejléc, saját hibaoldal, csak az `index.php` fut. Ismeretlen `APP_ENV` értékkel, hibás éles beállítással (debug, mindenkit megbízhatónak vevő `TRUSTED_PROXIES`, nem biztonságos süti, üres adatbázis- vagy Redis-jelszó) és hibás kimenő proxycímmel (`HDID_HTTP_PROXY`) a konténer nem indul.
A03 ellátási lánc, A08 integritás, ASVS V15	Minden alapimage digesttel, minden letöltés SHA-256-tal rögzítve, az alapimage Alpine-csomagjai a build idején a javított változatra frissítve; az nginx-forrás PGP-aláírással ellenőrzött; lockfájlos build, `npm ci --ignore-scripts`. Kiadás előtt: gitleaks, `composer audit`, `npm audit`, Hadolint, Trivy (javítható HIGH/CRITICAL leletnél a kiadás megáll). Aláírt image (cosign), build provenance és SBOM-igazolás; a CI action-jei commit-SHA-ra rögzítve.
A04 kriptográfia, ASVS V12	Kizárólag TLS 1.3, ML-KEM hibrid kulcscsere, HSTS, indításkori tanúsítvány-ellenőrzés; belső CA a tanúsítványtár mellé, a TLS-ellenőrzés nem kapcsolható ki.
Titkok, ASVS V13.3	Az image-ben nincs titok (gitleaks és Trivy titokkeresés). Titok fájlból (`*_FILE`); konfigurációs gyorsítótár csak memóriában; a migrációs adatbázisfiók jelszava csak a `mariadb` és a `migrate` szolgáltatásnál.
A09 naplózás, ASVS V16	Adatminimalizált JSON access log, forgatás, központi gyűjtés naplózó-meghajtóval, auditnapló az adatbázisban.
Nyitott tételek	A CSP-ben a Livewire/Alpine miatt `unsafe-inline` és `unsafe-eval` van. Egyéb nyitott tételek: az alternatív belépési utak és az MFA (az SSO-szolgáltatónál írható elő), az örökölt IVR URL-es kulcsa és módosító GET-hívásai. Az abszolút munkamenet-korlát, a fiókzárolás és a belépések naplózása a Beállítások › Munkatársi belépés és Ügyfélportál fülön állítható. Ezek értékelése telepítésenként, felelőssel és határidővel történjen.

---

13 Kipróbálás demóadatokkal

Tanúsítvány és külső rendszerek nélküli kipróbáláshoz a `.env` értékei:

```
APP_ENV=demo
APP_URL=https://localhost
OPENHDID_TLS=selfsigned
OPENHDID_DEMO_DATA=true
MAIL_MAILER=log
SMS_DRIVER=log
```

Az indítás után a migráció egyszer betölti a demóadatokat:

- ügyintéző: `agent@example.test`, jelszó `password-agent-1`;

- prémium ügyfél: `anna@example.test`, PIN 123456;
- normál ügyfél: `bela@example.test`;
- két hét hívástörténet.

Rendszergazdát a 3. fejezet 8. lépése szerint hozzon létre. A böngésző figyelmeztet az önaláírt tanúsítványra. A kimenő e-mailek és SMS-ek a `docker compose logs worker` kimenetében olvashatók. Demó módban a `hdid:health` kilépési kódja 2 (a `log` illesztők és a bekötetlen ügyféltörzs miatt), a `tls-check` önaláírt tanúsítványnál nem értelmezhető, a `/health` a gazdagépről a 6.6 pont szerint 404-et ad; a 12.2 pont többi próbája a `-k` kapcsolóval (önaláírt tanúsítvány) elvégezhető.

> **Csak kipróbáláshoz.**
>
> A demófiókok jelszava nyilvános. Demóadat csak `APP_ENV=demo` mellett töltődik be; éles környezetben (`APP_ENV=production`) a rendszer felismeri (a `.test` tartományú fiókok és ügyféltörzstételek, a demóhívások): a `migrate` 64-es kóddal, a talált címeket megnevező hibával megáll, ezért a `web`, a `worker` és az ütemező sem indul el; a demófiókok nem léphetnek be, és a Rendszerállapot „Demóadatok” kártyája piros. Kubernetesen az `openhdid-migrate` Job bukik. Éles telepítéshez új, üres adatbázissal kezdjen: a demókötetet élesítés előtt törölni kell, a feltöltött fájlokkal együtt (`docker compose down -v`), majd `docker compose up -d` és `hdid:make-admin`.

---

14 Frissítés új verzióra

Minden kiadás mellékleteként jön egy `UPGRADE.md`: ugyanezek a lépések másolható parancsokkal, az adott kiadás verziószámával kitöltve (letöltés ellenőrzőösszeggel, az új telepítési kulcsok listája, visszalépés).

1. Olvassa el a kiadási jegyzetet (GitHub Releases, `CHANGELOG.md`). Választhatóan ellenőrizze az image aláírását (üzemeltetési kézikönyv 8. fejezet).
2. Mentés: `./backup.sh` (adatbázis, `storage` kötet, beállítások, `tls`; üzemeltetési kézikönyv 7. fejezet). Ha a frissítés után vissza kell lépni: `./backup.sh restore`; utána mindenkinek újra be kell lépnie. Külső adatbázist a `backup.sh` nem ment: azt a kiszolgáló saját eszközével kell menteni a migráció előtt.
3. Az új kiadás fájljai ellenőrzőösszeggel (`SHA256SUMS`): a `compose.yaml`, a `backup.sh` és a használati profil fájlja, amelyre a `.env` `COMPOSE_FILE` sora mutat (az 5k profilnál a `profile-5k.yaml` melléklet a `profiles/5k/compose.yaml` helyre, a 10k profilnál a `profile-10k.yaml` a `profiles/10k/compose.yaml` helyre, mindkét gépen). A `.env`, a `db.env` és a `compose.override.yaml` marad. Az új `env.example`-ből a `.env`-ben még nem szereplő kulcsok átnézése: alapértékük van, csak akkor kell felvenni őket, ha a kiadási jegyzet kéri; az 1.7.0 előtti telepítésen a `COMPOSE_FILE` új kulcs, felvételéhez a profilfájl is kell.
4. Az `OPENHDID_VERSION` átírása az új verzióra, majd:

   ```
   docker compose pull
   docker compose up -d
   docker compose ps
   ```

A `migrate` lefuttatja az új migrációkat, utána a szolgáltatások az új image-dzsel indulnak újra. Egyetlen webpéldánynál ez néhány másodperces kiesés; kiesés nélküli frissítéshez több példány kell terheléselosztó mögött.

5. Ellenőrzés: `./install.sh check` (a 3. lépés tölti le az új `install.sh`-t is), majd a 12.1 pont. A frissítés után a `check` megnevezi a `.env`-ből hiányzó új (`ENV-01`) és a megszűnt (`ENV-02`) kulcsokat; az `./install.sh fix --only ENV-01,ENV-02` a hiányzókat a kommentjeikkel és alapértékükkel a `.env` végére fűzi, a megszűnteket kikommentezi, előtte a `.env`-et menti. Ugyanígy javítja a hiányzó profilfájlt (`FILES-04`) és a futás közben leállt ütemező beragadt zárát (`SCHED-01`).

Régi, adatbázisos alapértékű telepítés. Egy régebbi `env.example`-ből készült telepítés a munkameneteket, a gyorsítótárat és a sort az adatbázisban tartja (`SESSION_DRIVER` , `CACHE_STORE` , `QUEUE_CONNECTION` = `database`), miközben a beépített Valkey fut, gyakran jelszó nélkül. A `check` ezt `ENV-07` lelettel jelzi; az átállás `./install.sh fix --only ENV-07` : a `.env` mentése, `docker compose stop web scheduler` , a régi sor kiürülésének megvárása (`queue:monitor` , amíg 0), csak utána a `REDIS_PASSWORD` , a `redis` compose-profil és a három kulcs `redis` értéke, végül `docker compose up -d` . Egyszer mindenki kilép, a gyorsítótár egyszer üres; a `failed_jobs` tábla megmarad, adat nem törlődik; megszakításkor a `.env` változatlan marad. A kézi lépéseket az `UPGRADE.md` 4. lépése és a `TROUBLESHOOTING.md` 21. fejezete adja.

Két alkalmazásgép (10k) és Kubernetes (20k). A sorrendet (10k: mentés a migráció előtt, B gép, ellenőrzés, A gép, ellenőrzés mindkét gépen; 20k: a migrációs Job előbb, utána a gördülő frissítés), a két verzió együttfutásának feltételét és a visszalépést az `UPGRADE.md` „Két alkalmazásgép (10k profil)” és „Kubernetes (20k profil)” része írja le, másolható parancsokkal.

A platform frissítése (PHP, nginx, OpenSSL, Alpine) új javítóverzióként jelenik meg, a frissítés menete ugyanez. A kiadott címkék soha nem változnak, ezért egy telepítés mindig pontosan reprodukálható.

---

15 Használati profilok, magas rendelkezésre állás, Kubernetes

A telepítés három használati profil egyikére méretezhető; a 10k profil egy nagyobb gépen is futhat (15.7), és a két alkalmazásgépes változat közös adatbázisát és Valkey-jét egy infra gép is adhatja (15.8). Mindegyiket a telepítő (16. fejezet) állítja be. A profil nem új beállításréteg: a Compose-telepítésnél egy kiegészítő fájl a `compose.yaml` mellett (`profiles/5k/compose.yaml` , `profiles/10k/compose.yaml` , `profiles/10k/single.yaml` ; az infra gépé önálló: `profiles/10k/infra.yaml`), Kubernetesen egy Kustomize-overlay (`kubernetes/overlays/10k` , `kubernetes/overlays/20k`). A profil adja a PHP-FPM folyamatszámot, a feldolgozók számát, a beépített adatbázis méretét és a profil nevét (`HDID_SIZING_PROFILE`), amelyet a Rendszerállapot *Kapacitás* kártyája mutat: az utolsó 15 percben aktív fiókok számát, a mai csúcsot és a profil tervezett egyidejű felhasználószámát, 80 % fölött sárgán. Amikor a kártya sárgul, a következő profilra kell lépni.

A profilok feltevései: a portált az ügyfelek 5–8 %-a használja egyszerre (a tervezett érték a 8 %), az ügyintézők száma 20–30, naponta az ügyfelek 1–2 %-a hív. A munkamenet, a gyorsítótár és az üzenetsor mindhárom profilban Valkey-n fut, két példánnyal: a fő (`noeviction`) és a munkamenet-példány (`allkeys-lru`).

Kétszeres tartalék. A méretek a normál csúcsterhelés kétszeresére szólnak (normál csúcs profil nélkül kb. 25, 5k kb. 50, 10k kb. 75, 20k kb. 130 kérés/s; a méretezés ennek kétszerese): a normál csúcson legfeljebb kb. 50 % a kihasználtság, a másik fele a korlátokon belül maradó rosszindulatú terhelés tartaléka; ami ezen felül érkezik, azt a rendszer 429-cel vagy 503-mal elutasítja (6.8 pont). Minden szolgáltatásnak memóriaplafonja van, és a plafonok összege plusz 1 GB az operációs rendszernek belefér a gépbe. A térfogati DDoS a szolgáltató vagy egy előtét feladata.

15.1 A három profil

	`5K`	`10K`	`20K`
Ügyfél · egyidejű portálhasználó	5 000 · 250–400	10 000 · 500–800	20 000 · 1 000–1 600
Topológia	egy gép, Compose, beépített MariaDB, fő és munkamenet-Valkey	**magas rendelkezésre állás:** két alkalmazásgép Compose-zal terheléselosztó mögött, külső MariaDB (elsődleges + replika), két külső Valkey (fő és munkamenet), NFS a közös `storage` kötetnek; vagy Kubernetes (`overlays/10k`)	**Kubernetes:** web 3–6 pod (HPA), 3 feldolgozó, 1 ügyféltörzs-feldolgozó, 1 ütemező, migrációs Job; menedzselt MariaDB és két Valkey; ReadWriteMany kötet
Erőforrás	a lenti géptáblázat szerint		
PHP-FPM	12 folyamat, `dynamic` , 128M kérésenként	2 × 12, `static` (Kubernetesen podonként 8)	3–6 × 8 podonként, 2 nginx-dolgozó
MariaDB	`innodb_buffer_pool_size` 1G, 40 kapcsolat	4G, 80	8G, 120
Valkey	fő 256mb, munkamenet 384mb	fő 256mb (legalább 768 MB-os példány), munkamenet 576mb (legalább 1,25 GB)	fő 256mb (legalább 768 MB), munkamenet 1gb (legalább 2,25 GB)
Feldolgozók	2 `worker` , 1 `sync-worker` , 1 `scheduler`	1 `worker` gépenként; `sync-worker` és `scheduler` csak az elsődlegesen	3 `worker` , 1 `sync-worker` , 1 `scheduler`
Kiesés hatása	a gép kiesése a szolgáltatás kiesése	egy alkalmazásgép kiesése észrevétlen; adatbázis-átállás a replikára	egy pod vagy node kiesése észrevétlen
Fájl	`profiles/5k/compose.yaml`	`profiles/10k/compose.yaml` mindkét gépen, vagy `kubernetes/overlays/10k`	`kubernetes/overlays/20k`

A gépek mérete

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
Kipróbálás (profil nélkül, Compose vagy hagyományos)	Egy gép: web, feldolgozó, ügyféltörzs-feldolgozó, ütemező, MariaDB, két Valkey	1	2	8 GB	20 GB	8 PHP-FPM folyamat; a plafonok összege kb. 6,2 GB + 1 GB OS
5k (legfeljebb 5 000 ügyfél)	Egy gép: web, 2 feldolgozó, ügyféltörzs-feldolgozó, ütemező, MariaDB, fő Valkey és munkamenet-Valkey	1	4	10 GB	80 GB	12 PHP-FPM folyamat; MariaDB 1G puffer, 40 kapcsolat; fő Valkey 256mb (noeviction), munkamenet-Valkey 384mb (allkeys-lru); a plafonok összege kb. 8,8 GB + 1 GB OS
10k egy gépen (Compose)	Egy gép: web, 3 feldolgozó, ügyféltörzs-feldolgozó, ütemező, MariaDB, fő és munkamenet-Valkey	1	8	16 GB	120 GB	24 PHP-FPM folyamat; MariaDB 2G puffer, 60 kapcsolat; fő Valkey 256mb (noeviction), munkamenet-Valkey 576mb (allkeys-lru); a plafonok összege kb. 12,8 GB + 1 GB OS; nincs magas rendelkezésre állás
10k (Compose vagy hagyományos)	Alkalmazásgép terheléselosztó mögött (az elsődlegesen fut az ügyféltörzs-feldolgozó és az ütemező is)	2	4	6 GB	40 GB	gépenként 12 PHP-FPM folyamat; az elsődleges plafonjainak összege kb. 4,8 GB + 1 GB OS; egy gép kiesésekor a másik viszi a normál csúcsot

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
10k	Külső MariaDB, elsődleges + replika	2	2	8 GB	legalább 40 GB	innodb_buffer_pool_size 4G, max_connections 80 (a Kubernetes 10k-hoz is elég)
10k	Külső fő Valkey (sor, zárak, korlátok, gyorsítótár)	1	–	legalább 768 MB	–	maxmemory 256mb, maxmemory-policy noeviction (kötelező), AOF
10k	Külső munkamenet-Valkey (REDIS_SESSION_*)	1	–	legalább 1,25 GB	–	maxmemory 576mb, maxmemory-policy allkeys-lru (kötelező), RDB
10k	NFS a közös storage kötetnek	1	–	–	legalább 50 GB	arculati képek, feltöltések, auditarchívum
10k (Compose, infra gép)	Infra gép a külső MariaDB és a két külső Valkey helyett: MariaDB, fő és munkamenet-Valkey egy gépen	1	4	10 GB	100 GB	MariaDB 4G puffer, 80 kapcsolat (plafon 6g); fő Valkey 256mb (768m), munkamenet-Valkey 576mb (1280m); a plafonok összege 8 GB + 1 GB OS; egyetlen példány, a replika kézi lépés; a lemezben az itt exportált NFS 50 GB-ja
10k Kubernetes	Csomópontkészlet	3	4	8 GB	–	egy csomópont kiesésekor is elfér; MariaDB és a két Valkey a fenti 10k-sorok szerint (menedzselt)

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
10k Kubernetes	Podok (kérés = memóriakorlát; CPU-nak csak kérés van)	web 2–4 (HPA), feldolgozó 2, ügyféltörzs-feldolgozó 1, ütemező 1, migrate Job	web 1, feldolgozó 0,1, ügyféltörzs 0,25, ütemező 0,1, migrate 0,25	web 2 GiB, feldolgozó 384 MiB, ügyféltörzs 448 MiB, ütemező 1 GiB, migrate 384 MiB	RWX kötet 50 GB	web pod: 8 PHP-FPM folyamat, 2 nginx-dolgozó; ingress: 50 kérés/s, löket 200, 128 kapcsolat címenként
20k Kubernetes	Csomópontkészlet	3	4	12 GB	–	egy csomópont kiesésekor a HPA maximuma is elfér
20k Kubernetes	Podok (kérés = memóriakorlát)	web 3–6 (HPA), feldolgozó 3, ügyféltörzs-feldolgozó 1, ütemező 1, migrate Job	web 1, feldolgozó 0,1, ügyféltörzs 0,5, ütemező 0,1, migrate 0,25	web 2 GiB, feldolgozó 384 MiB, ügyféltörzs 448 MiB, ütemező 1 GiB, migrate 384 MiB	RWX kötet 50 GB	web pod: 8 PHP-FPM folyamat; a korlátok összege a HPA maximumán, frissítési többletpoddal kb. 17 GiB
20k	Menedzselt MariaDB	1 (+ replika)	4	16 GB	legalább 80 GB	innodb_buffer_pool_size 8G, max_connections 120
20k	Menedzselt fő Valkey	1	–	legalább 768 MB	–	maxmemory 256mb, noeviction (kötelező)
20k	Menedzselt munkamenet-Valkey	1	–	legalább 2,25 GB	–	maxmemory 1gb, allkeys-lru (kötelező)

PROFIL	GÉP VAGY KOMPONENS	DARAB	VCPU	RAM	LEMEZ	MEGJEGYZÉS
Minden profil	Tartalék	–	–	–	–	A méretek a normál csúcs kétszeresére szólnak: a normál csúcson legfeljebb kb. 50 % a kihasználtság, a másik fele a korlátokon (nginx- és ingress-korlát, útvonal- és azonosítónkénti korlátok) belül maradó rosszindulatú terhelés tartaléka. Ami ezen felül érkezik, azt a rendszer 429-cel vagy 503-mal elutasítja, nem omlik össze.
Minden profil	Térfogati (hálózati szintű, L3/L4) DDoS	–	–	–	–	Nem az alkalmazás feladata: a szolgáltató DDoS-szűrése vagy egy előtét (terheléselosztó, WAF) védi ki.

Számítási alap a saját méretezéshez (kemény felső határok, a kiadási fájlok ezekkel ellenőrzöttek): web = PHP-FPM folyamat × (`OPENHDID_FPM_MEMORY_LIMIT` 128M + 22 MB) + 208 MB OPcache + nginx-dolgozók × 80 MB + `/tmp` + 64 MB; CLI-folyamat (feldolgozó, migrate) = 256M + 22 MB + `/tmp` (az ütemező három folyamattal számol); MariaDB = puffer + 192 MB + `max_connections` × 20 MB; Valkey = 2 × `maxmemory` + 256 MB (fő, AOF), illetve + 128 MB (munkamenet, RDB); kapcsolatok: PHP-FPM × webpéldányok a HPA maximumán (+1 frissítési pod) + feldolgozók + 1 + 3 + 6 tartalék ≤ `max_connections`; a gépen a plafonok összege + 1 GB OS ≤ RAM, Kubernetes-csomóponton 1,5 GB foglalt, és egy kiesett csomópont mellett is elfér. A munkamenet-Valkey a normál csúcs × 0,85 KB × 7200 s vendégmunkamenetét és a valódi munkameneteket tartja, 20 % tartalékkal. A kérésenkénti PHP-CPU (20–30 ms) éles elrendezésen mérendő (üzemeltetési kézikönyv 9. fejezet).

15.2 5k profil: egy kiszolgáló

Ez az alaptelepítés a 3. fejezet szerint; a letöltött `env.example` már ezt a profilt választja, a profil fájlja a kiadás `profile-5k.yaml` melléklete, amelyet a 3. fejezet 1. lépése a `profiles/5k/compose.yaml` helyre tölt le. A `.env` idevágó sorai:

```
COMPOSE_FILE=compose.yaml:profiles/5k/compose.yaml
COMPOSE_PROFILES=db,redis
SESSION_DRIVER=redis
CACHE_STORE=redis
QUEUE_CONNECTION=redis
REDIS_HOST=valkey
REDIS_PASSWORD=…
```

A `REDIS_SESSION_HOST` itt üres maradhat: a `compose.yaml` a beépített Valkey mellett magától a `valkey-sessions` szolgáltatást adja. Ellenőrzés indítás után: `docker compose ps` két `worker` példányt és a `valkey-sessions` szolgáltatást mutatja; a Rendszerállapot *Kapacitás* kártyája a `5k profil` nevet és a 400-as tervezett értéket írja; a *Gyorsítótár* kártya `redis`. A profil értékei a `.env` azonos nevű soraival felülírhatók (`OPENHDID_FPM_MAX_CHILDREN`, `OPENHDID_DB_BUFFER_POOL`), ha a gép nagyobb vagy kisebb a tervezettnél.

15.3 Mikor kell lépni

- a *Kapacitás* kártya sárga: a mai csúcs eléri a profil tervezett egyidejű felhasználószámának 80 %-át, vagy az adatbázis-kapcsolatok 80 %-a használatban;
- a *Feladatsor* kártya tartósan sok várakozó üzenetet mutat: előbb egy további `worker` (`docker compose up -d --scale worker=3`), aztán a következő profil;
- a vezérlőpult vagy a portál válaszideje a mérésben (üzemeltetési kézikönyv 9. fejezet) a p95 300 ms fölé megy;
- rendelkezésre állási követelmény: 5 000 ügyfél fölött a kétgépes 10k profil akkor is indokolt, ha a terhelés még nem kívánná.

15.4 10k profil: két alkalmazásgép terheléselosztó mögött

A telepítővel: előbb az infra gép (`./install.sh --profile 10k-ha --role infra`, 15.8), utána az A és a B gép a válaszfájllal (`./install.sh --answers openhdid-10k-answers.txt --role a`, illetve `--role b`); saját külső adatbázis és Valkey mellett az A gép kérdezi a címeket, és ő írja a válaszfájlt (16.4). Az alábbi kézi leírás ugyanezt az elrendezést adja.

Előfeltételek. Két alkalmazásgép (A: elsődleges, B: másodlagos) azonos Docker-telepítéssel; terheléselosztó, amely a TLS-t lezárja és a két gép 80-as portjára oszt; külső MariaDB magas rendelkezésre állással (elsődleges + replika, egyetlen írható példány; a 7.2 pont két fiókjával); két külső Valkey jelszóval: a fő (gyorsítótár, sor, zárak, korlátszámlálók) `maxmemory 256mb` és kötelező `maxmemory-policy noeviction` mellett, AOF-naplóval (a sor tartósságához), legalább 768 MB-os példányon, és a munkamenet-tár `maxmemory 576mb` és kötelező `allkeys-lru` mellett, RDB-vel, legalább 1,25 GB-os példányon (a menedzselt szolgáltatások alapértéke gyakran `volatile-lru`; egy Valkey egy telepítésé, vagy legalább saját adatbázis, ACL-felhasználó és `REDIS_PREFIX`); NFS-export a közös `storage` kötetnek, a konténer felhasználója (uid 82) számára írhatóan. Az NFS tartja az arculati képeket, a Livewire ideiglenes feltöltéseit, a szinkronfájlokat és az auditarchívumot: enélkül a két webpéldány nem látja egymás fájljait.

A `.env` mindkét gépen (a 3. fejezet szerint, ezekkel az eltérésekkel; az `APP_KEY` azonos a két gépen; a profil fájlja a kiadás `profile-10k.yaml` melléklete, a `profiles/10k/compose.yaml` helyre töltve: `curl -fsSL --create-dirs -o profiles/10k/compose.yaml "$base/profile-10k.yaml"`):

```
COMPOSE_FILE=compose.yaml:profiles/10k/compose.yaml
COMPOSE_PROFILES=primary          # az A gépen; a B gépen üres
OPENHDID_TLS=off
TRUSTED_PROXIES=10.0.0.5          # a terheléselosztó címe(i)
OPENHDID_MONITORING_ALLOW=10.0.0.5   # az elosztó élő-próbája (/up)
DB_HOST=db.example.org            # külső MariaDB (írható elsődleges)
REDIS_HOST=valkey.example.org
REDIS_PASSWORD=…
REDIS_SESSION_HOST=valkey-sessions.example.org   # a munkamenetek Valkey-je
REDIS_SESSION_PASSWORD=…
REDIS_PREFIX=openhdid-eles-                    # mindkét gépen azonos, telepítésenként
egyedi
OPENHDID_NFS_SERVER=nfs.example.org
OPENHDID_NFS_PATH=/export/openhdid-storage
```

A `db.env` a migrációs fiókot adja mindkét gépen; a `mariadb` és a `valkey` szolgáltatás itt nem indul, mert a `db` és a `redis` compose-profil nincs megadva.

1. **A gép:** `docker compose up -d`. A `migrate` a közös Valkey zárja alatt fut, majd elindul a `web`, a `worker`, a `sync-worker` és a `scheduler`.
2. **B gép:** `docker compose up -d`. A `migrate` megvárja az A gép migrációját (zár), és nem változtat semmin; elindul a `web` és a `worker`. Ütemező és ügyféltörzs-feldolgozó itt nem fut: ezekből pontosan egy példány lehet.
3. **Terheléselosztó:** élő-próba a `http://<gép>:80/up` címen, a `Host` fejlécben az `APP_URL` gépnevével (más névre az nginx a kapcsolatot bontja, 444), és a terheléselosztó címével az `OPENHDID_MONITORING_ALLOW` listában (különben a `/up` válasza 404); a próba csak a PHP indulását mutatja; a `/health` a felügyeleti rendszereké, percenként 30 kérés egy címről (`OPENHDID_MONITORING_ALLOW`). A terheléselosztó a `X-Forwarded-For` és `X-Forwarded-Proto` fejlécet adja át; ragadós munkamenet nem kell, a munkamenetek a Valkey-ben közösek.
4. **Ellenőrzés:** mindkét gépen `docker compose ps`; a Rendszerállapot a `10k profil` nevet mutatja, a *Gyorsítótár* és a *Feladatsor* kártya `redis`; az access logban (`docker compose logs web`) az `ip` mező a kliens címe, nem a terheléselosztóé (különben a `TRUSTED_PROXIES` hibás).

Frissítés kiesés nélkül: az `UPGRADE.md` „Két alkalmazásgép (10k profil)” része szerint: mentés a migráció előtt (a külső MariaDB a kiszolgáló saját eszközével), a B gép frissítése (a `migrate` itt, a közös Valkey zárja alatt futtatja az új migrációkat), ellenőrzés, majd az A gép (a `migrate` itt már nem talál teendőt), végül ellenőrzés mindkét gépen. A két verzió a migráció és a második gép frissítése közti percekben együtt fut; a kiadási jegyzet „Frissítés előtt” sora mondja meg, ha egy kiadásnál ez nem megengedett.

Mentés: a `backup.sh` a beépített adatbázist menti, külső adatbázist nem (az `info.txt` `database=external` sora jelzi): a külső MariaDB-t a kiszolgáló saját eszközével (`mariadb-dump`, pillanatkép) kell menteni. A `storage` kötet NFS-en közös, ezért elég az A gépen menteni; a B gép saját `.env`-jét (`COMPOSE_PROFILES` `primary` nélkül), `db.env`-jét és `tls` mappáját ott kell menteni (a `./backup.sh` ott is futhat). Visszaállítás: előbb a B gépen `docker compose stop`, az adatbázist az üzemeltetője állítja vissza, utána az A gépen `./backup.sh restore <név>` és `./backup.sh signout` (ha az adatbázis a szkript után állt vissza), végül a B gépen `docker compose up -d`. A kiléptetés a közös Valkey munkamenet- és gyorsítótár-adatbázisát üríti, mindkét gépet érinti.

15.5 20k profil: Kubernetes

A telepítő (`./install.sh --profile 20k`, 16.6) a titok- és beállításfájlokat és egy saját overlayt készít a `k8s/` mappába, és kiírja az alábbi `kubectl` lépéseket; a fürtön semmit nem változtat.

Előfeltételek. Kubernetes-fürt (1.29 vagy újabb) ingress-vezérlővel (a manifeszt az ingress-nginx megjegyzéseit hordozza: `proxy-body-size`, címenként 50 kérés/s, 4-szeres löket és 128 kapcsolat, 120 mp-es proxy-időkorlát; az ingress-nginx a teljes IPv6-címre kulcsol, a /64-es kulcs a podokban él; a `HDID_RATE_LIMIT_EXEMPT_NETWORKS` csak a podok korlátja alól mentesít, a bejárat korlátja alól a `nginx.ingress.kubernetes.io/limit-whitelist` annotáció, ugyanazokkal a hálózatokkal, saját overlay-javítással; más vezérlőnél a megfelelőjüket kell megadni), NetworkPolicy-t ismerő hálózati bővítmény (Calico, Cilium, Antrea; a flannel önmagában nem): a `base/networkpolicy.yaml` a web podok 8080-as portját csak az `ingress-nginx` névtérből engedi, más ingress-vezérlőnél a névtér nevét az alkalmazás előtt saját overlay-javítással át kell írni, különben a portál elérhetetlen (ha a felügyelet közvetlenül a szolgáltatáson kérdezi a `/health` címet, a névtere második `namespaceSelector`-elemként vehető fel),, ReadWriteMany tárolóosztály (NFS, CephFS, Azure Files, EFS), menedzselt vagy operátorral futtatott MariaDB (a 7.2 pont két fiókjával) és Valkey; `kubectl` a kezelőgépen. A manifesztek a kiadás `kubernetes` mappájában vannak: a `base` a közös rész, az `overlays/10k` és `overlays/20k` a méret (replikák, erőforrások, PHP-FPM, a profil neve), az image címkéjét a kiadás írja a `base/kustomization.yaml`-ba.

A `.env` ugyanaz, mint a Compose-telepítésnél (`env.example`), ezekkel az eltérésekkel: `OPENHDID_TLS=off` (a TLS az ingressen zárul le), `TRUSTED_PROXIES` az ingress-vezérlő pod-hálózatának tartománya, `DB_HOST` és `REDIS_HOST` a menedzselt kiszolgálók címe (a fő Valkey-n `noeviction`), a `REDIS_SESSION_*` sorok a második, `allkeys-lru` politikájú Valkey címe, a `COMPOSE_*` és `OPENHDID_NFS_*` sorok nélkül. A `db.env` a migrációs fiókot adja.

1. **Névtér és titkok:**

   ```
   kubectl create namespace openhdid
   kubectl -n openhdid create secret generic openhdid-env --from-env-file=.env
   kubectl -n openhdid create secret generic openhdid-db-env --from-env-file=db.env
   kubectl -n openhdid create secret tls openhdid-tls --cert=fullchain.pem --
   key=privkey.pem
   kubectl -n openhdid create configmap openhdid-ca --from-file=ca/   # belső CA-k,
   elhagyható
   ```

2. **Saját kiszolgálónév:** az `overlays/20k/ingress.yaml` két `hdid.example.org` sorát az `APP_URL` gazdagépére kell írni (vagy saját overlayt készíteni, amely a `20k`-ra épül).
3. **Alkalmazás és migráció:**

   ```
   kubectl apply -k kubernetes/overlays/20k
   kubectl -n openhdid wait --for=condition=complete job/openhdid-migrate --timeout=600s
   kubectl -n openhdid get pods
   ```

A webpodok a migráció alatt is elindulnak, de a készenléti próbájuk (`openhdid healthcheck`, az FPM-en át) csak az induló alkalmazással teljesül; az élő-próba csak az nginx-et kérdezi (`openhdid healthcheck live`), így egy terhelt pool miatt a pod nem indul újra. A podok memóriakérése egyenlő a korlátjukkal (web 2Gi, ütemező 1Gi), a csomópontok méretét a fenti géptáblázat adja; a Job a `db.env` migrációs fiókjával fut, a podok a futási fiókkal.

4. **Ellenőrzés:** `kubectl -n openhdid logs deploy/openhdid-web`; a Rendszerállapot `20k profil` nevet mutat; `kubectl -n openhdid get hpa` a web skálázását.

Frissítés: az `UPGRADE.md` „Kubernetes (20k profil)” része szerint. Az `OPENHDID_VERSION` megfelelője itt a `base/kustomization.yaml` `newTag` sora: a manifesztek nem kiadási mellékletek, hanem a kiadás címkéjéből jönnek, és abban a `newTag` már a kiadás verziója. Sorrend: a régi migrációs Job törlése, a migrációs Job futtatása és megvárása, csak utána a teljes `kubectl apply -k`, amely a webpodokat gördülő frissítéssel cseréli (`maxUnavailable: 0`), az ütemezőt és az ügyféltörzs-feldolgozót `Recreate` stratégiával, egy példányban. Visszalépés: migráció nélkül a `kubectl rollout undo`; migrációval az adatbázis visszaállítása a frissítés előtti mentésből, mindenki kiléptetése még a podok indítása előtt, végül a régi verzió manifesztjei (a migrációs Job törlése után). Frissítés előtt mentés: az adatbázis pillanatképe a menedzselt szolgáltatás eszközével.

Biztonsági környezet. Minden pod nem root (uid 82), csak olvasható gyökér-fájlrendszerrel, minden Linux-képesség nélkül, `RuntimeDefault` seccomp-profillal; a `/tmp` memóriában lévő `emptyDir`. A PodDisruptionBudget két webpodot tart meg karbantartáskor; a webpodok lehetőleg külön node-okra kerülnek.

Mentés: az adatbázist a menedzselt szolgáltatás eszközével, a `storage` kötetet a tárolóosztály pillanatképével vagy fájlszintű mentéssel; a titkokat (`.env`, `db.env`, tanúsítvány) a fürtön kívül, védett helyen.

15.6 Mérés

A helyes méret mérésből jön. A kiadás nem tartalmaz mérőeszközt; az üzemeltetési kézikönyv 9. fejezete írja le, mit és milyen küszöbbel kell mérni (k6 vagy hasonló eszközzel, a profil forgatókönyveivel), és mikor kell a következő profilra lépni.

15.7 10k profil egy gépen

Legfeljebb 10 000 ügyfél (500–800 egyidejű portálhasználó) egyetlen, nagyobb gépen: 8 vCPU, 16 GB RAM, 120 GB lemez, beépített MariaDB és két Valkey. A méretezés a 10k normál csúcsának (kb. 75 kérés/s) kétszeresére szól, egy gépen. **Nincs magas rendelkezésre állás:** a gép kiesése a szolgáltatás kiesése; ha ez nem elfogadható, a kétgépes 10k (15.4) vagy a Kubernetes (15.5) a választás. A profil fájlja a kiadás `profile-10k-single.yaml` melléklete a `profiles/10k/single.yaml` helyre; a telepítő (menü: „10k egy gépen”, `--profile 10k-single`) tölti le és állítja be. Kézzel a 3. fejezet szerint, ezekkel a sorokkal:

```
curl -fsSL --create-dirs -o profiles/10k/single.yaml "$base/profile-10k-single.yaml"
COMPOSE_FILE=compose.yaml:profiles/10k/single.yaml
COMPOSE_PROFILES=db,redis
```

SZOLGÁLTATÁS	MÉRET	MEMÓRIAPLAFON
`web`	24 PHP-FPM folyamat (`dynamic` , 8 induló és tartalék, legfeljebb 16 tartalék), 128M kérésenként, 4 nginx-dolgozó	4480m (24 × 150 + 208 + 4 × 80 + 256 + 64 MB)
`worker`	3 példány	3 × 384m
`sync-worker` , `scheduler` , `migrate`	1–1	448m, 1g, 384m
`mariadb`	`innodb_buffer_pool_size` 2G, 60 kapcsolat (24 + 3 + 1 + 3 + 6 = 37 kell)	3584m
`valkey`	256mb, `noeviction` , AOF	768m
`valkey-sessions`	576mb, `allkeys-lru` , RDB	1280m
Összesen	a plafonok összege 13 120 MB (kb. 12,8 GB) + 1 GB az operációs rendszeré	≤ 16 GB, kb. 2,2 GB marad a lapgyorsítótárnak

A Rendszerállapot *Kapacitás* kártyája a „10k profil (egy kiszolgáló)” nevet és 800 tervezett egyidejű felhasználót mutat (`HDID_SIZING_PROFILE=10k-single`). A `.env` -ben felülírt `OPENHDID_FPM_*` , `OPENHDID_DB_*` , `OPENHDID_WEB_MEM_LIMIT` és `OPENHDID_VALKEY_*` értékek az itteni alapértékeket írják felül; a plafont a hozzá tartozó értékkel együtt kell emelni.

15.8 Infra gép a kétgépes 10k-hoz

Ha nincs saját, magas rendelkezésre állású MariaDB és két Valkey, a kétgépes 10k közös szolgáltatásait egy harmadik gép adja: 4 vCPU, 10 GB RAM, 100 GB lemez (benne az itt exportált NFS 50 GB-ja). A fájl a kiadás `profile-10k-infra.yaml` melléklete (`profiles/10k/infra.yaml` , önálló Compose-fájl, projektnév `openhdid-infra`); a telepítő (`./install.sh --profile 10k-ha --role infra` , 16.4) írja a gép saját, csak az infra kulcsait tartalmazó `.env` -jét és a `db.env` -et, indítja és ellenőrzi a szolgáltatásokat, és a közös értékeket az A és a B gépnek a válaszfájlba teszi.

- **MariaDB:** a `compose.yaml` -lal azonos image, `innodb_buffer_pool_size` 4G, 80 kapcsolat (2 × 12 PHP-FPM, a feldolgozók, az ütemező, a migráció és tartalék), plafon 6g; a futási és a migrációs fiók az első induláskor jön létre (7.2).
- **Fő Valkey:** `maxmemory 256mb` , `maxmemory-policy noeviction` (kötelező), AOF, plafon 768m, jelszó: `REDIS_PASSWORD` .
- **Munkamenet-Valkey:** `maxmemory 576mb` , `allkeys-lru` , RDB, plafon 1280m, saját jelszó: `REDIS_SESSION_PASSWORD` (itt a két Valkey hálózaton érhető el, ezért külön jelszavuk van).
- **Portok:** a gép privát címén (`OPENHDID_INFRA_BIND_IP` , a `0.0.0.0` -ra a `check` figyelmeztet: `INFRA-03`) 3306, 6379, 6380 (`OPENHDID_INFRA_DB_PORT` , `_VALKEY_PORT` , `_VALKEY_SESSIONS_PORT`).
- **Egyetlen példány:** amíg nincs replika, az infra gép kiesése a szolgáltatás kiesése.

Tűzfal (a telepítő ugyanezt írja ki a megadott A és B címmel; csak javasolja, nem futtatja): a 3306-os, 6379-es, 6380-as és (NFS-nél) 2049-es port csak az A és a B gép felől.

```
sudo firewall-cmd --permanent --add-rich-rule='rule family=ipv4 source address=10.0.0.11 port port=3306 protocol=tcp accept'
sudo firewall-cmd --permanent --add-rich-rule='rule family=ipv4 source address=10.0.0.12 port port=3306 protocol=tcp accept'
# … ugyanez a 6379-es, a 6380-as és a 2049-es portra
sudo firewall-cmd --reload
sudo ufw allow from 10.0.0.11 to any port 3306 proto tcp      # ufw-vel ugyanez
```

NFS-export a közös `storage` kötetnek (root kell; a telepítő csak kiírja). A könyvtár a konténer felhasználójáé (82); rootless Dockerrel futó alkalmazásgépen a leképezett azonosító (a `/etc/subuid` kezdő értéke + 81, 3. fejezet „Rootless Docker”), de a rootless démon NFS-kötetet nem csatol (16.4):

```
sudo mkdir -p /export/openhdid-storage
sudo chown 82:82 /export/openhdid-storage
sudo chmod 750 /export/openhdid-storage
echo '/export/openhdid-storage 10.0.0.11(rw,sync,no_subtree_check,root_squash) 10.0.0.12(rw,sync,no_subtree_check,root_squash)' | sudo tee -a /etc/exports
sudo exportfs -ra
showmount -e 10.0.0.20          # az A és a B gépen: az export látszik
```

Mentés: a `backup.sh` az alkalmazásgépé, itt nem fut (nincs `compose.yaml` és `web`). Az adatbázis mentése a konténer környezetéből vett jelszóval (a jelszó nem kerül a parancssorba):

```
docker compose exec -T mariadb sh -c 'MYSQL_PWD="$DB_PASSWORD" exec mariadb-dump -u"$DB_USERNAME" --single-transaction --no-tablespaces --default-character-set=utf8mb4 "$MARIADB_DATABASE"' | gzip > openhdid-$(date +%F).sql.gz
```

MariaDB-replika (a telepítő nem készíti el; lépésenként kiírja):

1. Második gép ugyanezzel az infra-fájllal és telepítéssel (a saját `.env`-jében más `OPENHDID_INFRA_BIND_IP`).
2. A bináris napló mindkét gépen egy `compose.override.yaml`-ban, a `COMPOSE_FILE` végére fűzve: `services: mariadb: command: [--server-id=1, --log-bin=mariadb-bin, --binlog-format=ROW]` (a replikán `--server-id=2`), majd `docker compose up -d`.
3. Replikációs fiók az elsődlegesen (`docker compose exec mariadb mariadb -uroot -p`): `CREATE USER 'repl'@'<a replika címe>' IDENTIFIED BY '<REPL_JELSZÓ>';` és `GRANT REPLICATION SLAVE ON *.* TO 'repl'@'<a replika címe>';`.
4. Konzisztens kiindulópont: az elsődlegesen `docker compose exec -T mariadb sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" exec mariadb-dump -uroot --all-databases --single-transaction --master-data=2 --gtid' | gzip > replika-alap.sql.gz`, a replikán `gunzip -c replika-alap.sql.gz | docker compose exec -T mariadb sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" exec mariadb -uroot'`.
5. A replikán: `CHANGE MASTER TO MASTER_HOST='10.0.0.20', MASTER_PORT=3306, MASTER_USER='repl', MASTER_PASSWORD='<REPL_JELSZÓ>', MASTER_USE_GTID=slave_pos;`, `START SLAVE;`, ellenőrzés: `SHOW SLAVE STATUS\G` (`Slave_IO_Running` és `Slave_SQL_Running`: `Yes`).
6. Átállás kézzel: a `DB_HOST` az A és a B gép `.env`-jében a replika címére, utána mindkét gépen `docker compose up -d`.

Ellenőrzés az infra gépen: `./install.sh check` (a három szolgáltatás állapota: `INFRA-01`; a két Valkey politikája és korlátja: `INFRA-02`; a figyelő cím: `INFRA-03`; a két adatbázis-fiók jogai: `INFRA-04`).

---

16 A telepítő (install.sh)

Az `install.sh` minden használati profil egyetlen belépési pontja: a 3. fejezet kézi lépéseit, a 15. fejezet profilváltozatait és a kétgépes 10k három gépét ugyanazzal a paranccsal, kérdések alapján telepíti, utána ellenőrzi, és a hibakeresésben és a javításban is vezet. A kiadás melléklete és a nyilvános repó gyökerének eleme; a telepítés mappájába kerül, és mindig a saját mappájából dolgozik.

```
sudo mkdir -p /opt/openhdid && sudo chown "$(id -un)" /opt/openhdid && cd /opt/openhdid
base=https://github.com/adamfhun/openhdid/releases/latest/download
curl -fsSL -O "$base/install.sh" -O "$base/SHA256SUMS"
sha256sum -c --ignore-missing SHA256SUMS      # install.sh: OK
chmod 755 install.sh
./install.sh
```

16.1 Mit csinál, és mit nem

- **Átlátható:** induláskor nyelvet kérdez (`--lang hu|en`), minden lépést számozva ír ki (`==> [3/12] A kiadás fájljai`), alatta azt, miért fut, és a végén, mi lett; a végén összefoglaló (profil, mappa, projekt, verzió, a megírt fájlok jogai, az ellenőrzés eredménye) és számozott „Következő teendők” lista jön, pontos parancsokkal és dokumentum-hivatkozással. A napló a `.install/install-<időbélyeg>.log` fájlban marad, titok nélkül.
- **A kiadás fájljai** a választott kiadásból (alapból a szkript saját kiadása) jönnek, és csak egyező `SHA256SUMS`-sorral kerülnek a helyükre; eltérésnél semmi nem változik. A futó `install.sh`-t is összeveti a kiadás ellenőrzőösszegével.
- **Titkok:** az `APP_KEY` és a jelszavak az alkalmazás image-ével, hálózat nélküli egyszeri konténerben készülnek (az infra gépen a gazdagép `/dev/urandom`-jából); a képernyőn és a naplóban csak a hosszuk látszik („generált, 48 karakter”), parancssori argumentumba nem kerülnek. A `.env` és a `db.env` 600-as joggal készül, az `env.example` minden megjegyzése megmarad.
- **Meglévő telepítésre nem telepít:** ha a mappában már van `.env` és a projektnek konténere vagy kötete, kilép (kód 2), és a `check`-re, a `fix`-re és az `UPGRADE.md`-re mutat. Félbemaradt telepítést (van `.env`, de konténer nincs) megerősítés és `.env.bak-<időbélyeg>` mentés után folytat a meglévő kulcsokkal; ha a telepítő a `.env` megírása után, de az indítás előtt állt meg (például a tanúsítvány ellenőrzésén), a saját egyszeri konténerei által létrehozott, még üres kötetek mellett is (ezt a `.install/progress` fájl jelzi, az indítás után a `.install/state` dönt). Meglévő adatbázis- vagy `storage`-kötet mellett `APP_KEY`-t soha nem generál.
- **Tanúsítvány:** az új párt a konténer a helyére tétel előtt ellenőrzi (kulcspár, lánc, név, lejárat), az élő `tls` mappa mellé csatolva; hibás pár nem kerül a helyére. A `tls` és a `ca` mappa 755-ös joggal jön létre (a konténer felhasználója a gazdagépen „mások” közé tartozik); a `ca` mappába tett CA-fájl 644-es legyen, különben a konténer induláskor nem veszi fel (`TLS-05`).
- **Jelszavak:** a válaszfájlban megadott jelszó a `.env`-be aposztrófok közé kerül, így a `$`, a szóköz, a `#` és a visszaperjel szó szerint marad; aposztróf és sortörés nem lehet benne. A beépített Valkey jelszavában idézőjel és visszaperjel sem, mert a Valkey a jelszót a Compose-tól idézőjelek között kapja.

- **Soha:** nem telepít csomagot, nem nyúl a tűzfalhoz, a rendszer és a Docker-démon beállításaihoz (ezekre csak a parancsot írja ki, `sudo`-val), nem kér rootot (rootként futtatva az `install` és a `fix` megtagad), nem futtat `--user 0` konténert, adatot nem töröl, semmit nem tölt fel. A tanúsítványt a konténer felhasználója teszi a helyére a `backup.sh` visszaállításának módszerével, így rootless Dockerrel is a helyes tulajdonos áll be (a 3. fejezet „Rootless Docker” részének kézi `chown`-ja nem kell).
- **Kilépési kód:** 0 kész, és az ellenőrzés tiszta; 1 kész, figyelmeztetéssel; 2 megállt (kiírja, meddig jutott, és mivel folytatható).

Kapcsolók: `--profile trial|5k|10k-single|10k-ha|20k` és `--role a|b|infra` a menü helyett; `--answers FÁJL` válaszfájl (csak a tulajdonos olvashatja, különben `chmod 600` tanáccsal elutasítja; csak ismert kulcsok, `KULCS=érték` sorok, a szkript semmit nem értékel ki belőle); `--from MAPPA` letöltés helyett helyi kiadási fájlok (a mellékletek vagy a repó fája; `SHA256SUMS` nélkül figyelmeztet); `--version X.Y.Z`; `--yes` minden megerősítést igennek vesz, csak válaszfájllal, próbához és füstteszthez, élesre nem ajánlott (az első rendszergazdát ilyenkor nem hozza létre, a parancsot a teendők közé írja).

16.2 Profil és szerep

VÁLASZTÁS	MIT TELEPÍT	A GÉP (15. FEJEZET)	A `.ENV` PROFILSORAI
1) Kipróbálás	profil nélkül, önaláírt tanúsítvánnyal, demóadatokkal (13. fejezet)	2 vCPU, 8 GB, 20 GB	`COMPOSE_FILE` kikommentelve, `COMPOSE_PROFILES=db,redis`, `APP_ENV=demo`
2) 5k egy gépen (alapértelmezés)	a 15.2 pont telepítése	4 vCPU, 10 GB, 80 GB	`COMPOSE_FILE=compose.yaml:profiles/5k/compose.yaml`, `COMPOSE_PROFILES=db,redis`
3) 10k egy gépen	a 15.7 pont telepítése, magas rendelkezésre állás nélkül	8 vCPU, 16 GB, 120 GB	`COMPOSE_FILE=compose.yaml:profiles/10k/single.yaml`, `COMPOSE_PROFILES=db,redis`
4) 10k két alkalmazásgéppel, szerep `infra`	MariaDB a két fiókkal, fő és munkamenet-Valkey (15.8)	4 vCPU, 10 GB, 100 GB	`COMPOSE_FILE=profiles/10k/infra.yaml`
4) … szerep `a`	elsődleges alkalmazásgép: web, feldolgozó, ütemező, ügyféltörzs-feldolgozó, a migráció (15.4)	4 vCPU, 6 GB, 40 GB	`COMPOSE_FILE=compose.yaml:profiles/10k/compose.yaml`, `COMPOSE_PROFILES=primary`, `OPENHDID_TLS=off`
4) … szerep `b`	másodlagos alkalmazásgép: web és feldolgozó	4 vCPU, 6 GB, 40 GB	mint az A, de `COMPOSE_PROFILES` üres
5) 20k Kubernetes	titok- és beállításfájlok és saját overlay a `k8s/` mappában, kiírt `kubectl` lépések (16.6)	csomópontkészlet 3 × (4 vCPU, 12 GB)	–

A választás után a telepítő kiírja a géptáblázat sorát és a profil lényegét (meddig elég, mi a kiesés hatása), majd ellenőrzi a gépet: processzormag, memória (a sor 90 %-a), szabad lemez a mappában és a Docker adatmappájában, idő-szinkron, a portok szabadsága, rootless Dockernél az 1024 alatti portok, a Docker Engine (27+) és a Compose (2.24+) verziója, Podman (nem támogatott), a démon proxyja. Hibánál megáll, figyelmeztetésnél megerősítést kér; az azonosítók (`HOST-*`, `DOCKER-*`) ugyanazok, mint a `check`-ben (16.7). Ha a mappában `compose.override.yaml` van, a `COMPOSE_FILE` végére fűzi (a Compose a `COMPOSE_FILE` mellett magától nem olvassa).

16.3 Egy gép: kipróbálás, 5k, 10k egy gépen

A kérdések sorrendje (szögletes zárójelben az alapérték):

- **kipróbálás:** HTTPS-port [443], HTTP-port [80], figyelő cím [0.0.0.0], a webhely címe [`https://<a gép neve>`, a 443-tól eltérő portot beleírja], demóadat [true], környezet [demo]; a tanúsítvány önaláírt;
- **5k és 10k egy gépen:** portok és figyelő cím, a webhely nyilvános címe (`APP_URL`, kötelező, a DNS-ben ezzel a névvel), környezet [production], TLS [on; off = TLS-t lezáró terheléselosztó mögött, ekkor a terheléselosztó címe(i) a `TRUSTED_PROXIES` és az `OPENHDID_MONITORING_ALLOW` sorba; selfsigned csak próbára], `on` módban a teljes lánc és a kulcs útvonala [`./tls/fullchain.pem`, `./tls/privkey.pem`, ha ott vannak], a kéréskorlát alól mentesített hálózatok [üres; elemenként legfeljebb /8], az első rendszergazda e-mail-címe [üres = később] és neve;
- ha a gépen már van ilyen nevű Compose-projekt vagy kötet, egy új projektnevet is kér (`COMPOSE_PROJECT_NAME`; több telepítés egy gépen).

Az összegzés minden értéket megmutat (a titkokat csak hosszal), felsorolja a megváltozó fájlokat, és kimondja, hogy az első indításkor lefutnak a migrációk; csak megerősítés után folytatja. A lépések: a gép, kérdések, a kiadás fájljai, image-ek, titkok, a `.env` és a `db.env`, tanúsítvány (`tls-check`-kel), a konfiguráció ellenőrzése (`docker compose config` és a konténer saját indulási ellenőrzése, `check-env`), indítás (`docker compose up -d --wait`; hibánál a hibás szolgáltatás naplójának kitakart vége és a `TROUBLESHOOTING.md` 3. fejezete), ellenőrzés (`install.sh check`), az első rendszergazda (a jelszót a `hdid:make-admin` maga kéri rejtve, csak terminálon), összefoglaló.

A végén a következő teendők: a DNS-rekord, a tanúsítvány megújítása (`./install.sh fix --only TLS-02`, vagy a 6.3 pont szerint és `docker compose exec web openhdid reload`), a tűzfal (firewalld és ufw példa, csak a szükséges portokkal), a levelezés, az SMS és az ügyféltörzs bekötése (11. fejezet; addig a `check` „még nincs bekötve” figyelmeztetést ad), a telefonközpont kulcsa, a napi mentés a saját crontabba (`30 2 * * * /opt/openhdid/backup.sh --quiet`), és a rendszeres `./install.sh check`.

16.4 Két alkalmazásgép és infra gép (10k)

A sorrend: **infra** → **A** → **B**. Saját, külső MariaDB és Valkey mellett nincs infra gép: ekkor az A gép kérdezi a külső címeket és jelszavakat, és ő írja a válaszfájlt.

1. **Infra gép:** `./install.sh --profile 10k-ha --role infra`. Kérdések: figyelő cím [a gép első privát IPv4-címe; a 0.0.0.0-ra figyelmeztet, mert a MariaDB és a Valkey itt titkosítatlanul beszél], az infra gép címe az A és a B felől, a három port [3306, 6379, 6380], az A és a B gép címe (a tűzfal- és NFS-parancsokhoz), az adatbázis neve és a két fiók [openhdid, openhdid_migrate], a webhely címe és a terheléselosztó címe (üresen az A gép kérdezi), az NFS-kiszolgáló [ez a gép] és útvonal [`/export/openhdid-storage`], a kulcselőtag [`openhdid-<6 hexa>-`]. Elindítja a három szolgáltatást (`profiles/10k/infra.yaml`, a gép saját, magyar megjegyzéses `.env`-jével), ellenőrzi őket, és

megírja az `openhdid-10k-answers.txt` válaszfájlt (600): `APP_KEY` (itt készül), a címek és portok, a futási és a migrációs fiók, a két Valkey jelszava, a kulcselőtag, az `APP_URL`, a terheléselosztó és az NFS adatai. Kiírja a tűzfal, az NFS-export, a mentés és a replika lépéseit (15.8).

2. **A válaszfájl átvitele:** `scp openhdid-10k-answers.txt install.sh <A>:/opt/openhdid/`, ugyanez a B-re, a célgépen `chmod 600 openhdid-10k-answers.txt`. A fájl minden közös titkot tartalmaz: használat után mindhárom gépen törlendő (`rm openhdid-10k-answers.txt`); amíg ott van, a `check` figyelmeztet (`FILES-08`).
3. **A gép:** `./install.sh --answers openhdid-10k-answers.txt --role a`. Csak a hiányzó közös értékeket kérdezi (és megerősítéssel a fájlba is visszaírja), utána a HTTP-portot [80], a figyelő címet, a mentesített hálózatokat és az első rendszergazdát. A `migrate` itt fut a közös Valkey zárja alatt; az ütemező és az ügyféltörzs-feldolgozó csak itt indul (`COMPOSE_PROFILES=primary`).
4. **B gép:** `./install.sh --answers openhdid-10k-answers.txt --role b`. Csak a szerepet veszi tudomásul; minden más a válaszfájlból jön (hiányzó kötelező értéknél megáll: a fájlt az infra vagy az A gép telepítője írja). A `migrate` itt nem talál teendőt („Nothing to migrate”). A sikeres telepítés után megkérdezi, törölje-e a válaszfájlt (alapból igen; `--yes` mellett nem töröl, csak figyelmeztet).
5. **Terheléselosztó, DNS, NFS-ellenőrzés:** az A és a B gép kimenete a HAProxy-mintát (6.5), a DNS-rekordot (`<gép> → <terheléselosztó>`), a tűzfalsort (a HTTP-port csak a terheléselosztó felől) és a `showmount -e <NFS>` próbát írja ki.

Rootless Dockerrel az alkalmazásgépek démona NFS-kötetet nem csatol: a telepítő ezt hibaként jelzi (`DOCKER-08`); megoldás a rootful Docker (`docker` csoport), vagy a gazdagépen csatolt NFS és egy `compose.override.yaml` bind kötete (mindkettő root-feladat, a telepítő egyiket sem végzi el).

16.5 A válaszfájl

Az első sor `# openhdid-answers 1`, utána `KULCS=érték` sorok, `#` megjegyzés és üres sor. A szkript soronként olvassa, semmit nem értékel ki (nincs `source`, `eval`, kifejtés), csak az ismert kulcsokat fogadja el; ismeretlen vagy ismétlődő kulcsnál, rossz formátumnál a sor számával megáll. A gépi telepítéshez (próba, füstteszt) minden kérdés kulcsa megadható, például:

```
# openhdid-answers 1
ANSWERS_FORMAT=1
PROFILE=5k
APP_URL=https://hdid.example.org
APP_ENV=production
TLS_MODE=on
TLS_CERT=/opt/certs/fullchain.pem
TLS_KEY=/opt/certs/privkey.pem
EXEMPT_NETWORKS=
ADMIN_EMAIL=
```

Ismert kulcsok: `ANSWERS_FORMAT`, `OPENHDID_VERSION`, `PROFILE`, `ROLE`, `PROJECT_NAME`, `APP_ENV`, `APP_URL`, `TLS_MODE`, `TLS_CERT`, `TLS_KEY`, `BIND_IP`, `HTTPS_PORT`, `HTTP_PORT`, `EXTRA_HOSTS`, `EXEMPT_NETWORKS`, `LB_ADDRESSES`, `APP_HOSTS`, `DEMO_DATA`, `APP_KEY`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, `DB_MIGRATE_USERNAME`, `DB_MIGRATE_PASSWORD`, `MARIADB_ROOT_PASSWORD`, `REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_SESSION_HOST`, `REDIS_SESSION_PORT`, `REDIS_SESSION_PASSWORD`, `REDIS_PREFIX`, `NFS_SERVER`, `NFS_PATH`, az infra gépé (`INFRA_BIND_IP`, `INFRA_ADDRESS`, `INFRA_DB_PORT`, `INFRA_VALKEY_PORT`, `INFRA_VALKEY_SESSIONS_PORT`), a Kubernetesé (`K8S_NAMESPACE`, `K8S_INGRESS_NAMESPACE`, `K8S_POD_CIDR`, `K8S_STORAGE_CLASS`) és az első rendszergazdáé (`ADMIN_EMAIL`, `ADMIN_NAME`). A jelszavak 12–256 karakteresek, aposztróf és sortörés nélkül.

16.6 Kubernetes (20k)

Kérdések: névtér [openhdid], `APP_URL`, az ingress-vezérlő névtere [ingress-nginx] és pod-hálózata (`TRUSTED_PROXIES`, nem lehet „mindenki”), a külső MariaDB és a két Valkey címe, fiókja és jelszava, a kulcselőtag, a tárolóosztály, a tanúsítvány és a kulcs útvonala, a belső CA-k mappája. A telepítő a `k8s/` mappába (700) írja az `openhdid-env.env` és `openhdid-db-env.env` fájlt (600, idézőjel nélkül, mert a `kubectl create secret --from-env-file` az idézőjelet az érték részének venné), és egy `kustomization.yaml`-t a kiadás overlayére (`overlays/20k`) a képcímkével, a kiszolgálónévvel, az eltérő ingress-névtérrel és a tárolóosztállyal. A fürtön semmit nem változtat (ha van `kubectl`, csak a helyi `kubectl kustomize k8s` fut); a lépéseket (15.5) kiírja és a `k8s/LEPESEK.txt` fájlba is menti: névtér, a két titok és a tanúsítvány, a migrációs Job, majd `kubectl apply -k k8s` és a `rollout status`, végül az első rendszergazda.

16.7 Ellenőrzés, javítás, diagnosztikai csomag

```
./install.sh check            # teljes ellenőrzés; kilépési kód 0 rendben, 1
figyelmeztetés, 2 hiba
./install.sh check --json     # ugyanez gépnek (openhdid-check/1)
./install.sh fix              # vezetett javítás, leletenként megerősítéssel
./install.sh fix --only ENV-07,SCHED-01
./install.sh report           # kitakart diagnosztikai csomag a report/ mappában (600)
```

A `check` csak olvas: a gépet, a Dockert, a fájlokat és jogaikat, a `.env`-et a kiadás `env.example`-jéhez képest (a frissítés utáni új és megszűnt kulcsok is), a konténer indulási ellenőrzését, a tanúsítványt, a szolgáltatások állapotát, a `hdid:health` csempéit, az adatbázis-fiókokat, a Valkey jelszavát és kötelező politikáját, a nevek feloldását, a beragadt ütemező-zárat és a naplók ismert hibáit nézi. Minden lelet azonosítót, súlyosságot, okot, teendőt (pontos paranccsal, javítható leletnél `./install.sh fix --only <azonosító>`) és a `TROUBLESHOOTING.md` fejezetét kapja; a fejezetek a lelet-azonosítókat is megnevezik, a 24. fejezet a teljes mutató. A `fix` és a `report` használatát és a leletek olvasását az üzemeltetési kézikönyv 6. fejezete írja le.

Rövidítésjegyzék

RÖVIDÍTÉS	JELENTÉS
EMD	Enterprise Master Data; magyarul ügyféltörzs. A munkatársi és ügyféladatok közös külső törzsadatforrása; a parancsok (`hdid:emd-sync`) és a telepítési kulcsok (`EMD_SYNC_*`) viselik a rövidítést.
ACME	Automatic Certificate Management Environment – automatikus tanúsítványkiadási protokoll.
API	Application Programming Interface – gépi alkalmazási felület.
ASVS	Application Security Verification Standard – tételes alkalmazásbiztonsági ellenőrzési követelmények (OWASP).

RÖVIDÍTÉS	JELENTÉS
CA	Certificate Authority – tanúsítványkiadó.
CSP	Content Security Policy – a böngészőben betölthető és futtatható tartalom szabályzata.
FPM	FastCGI Process Manager – a PHP folyamatkezelője.
GHCR	GitHub Container Registry – a GitHub konténer-image tára.
HSTS	HTTP Strict Transport Security – a böngészőt HTTPS használatára kötelező szabály.
ML-KEM	Module-Lattice Key Encapsulation Mechanism – poszt-kvantum kulcscsere-eljárás (FIPS 203).
OIDC, SSO	OpenID Connect; Single Sign-On – egyszeri, vállalati bejelentkezés.
OWASP	Open Worldwide Application Security Project – nyílt alkalmazásbiztonsági közösség és útmutatók.
SAN	Subject Alternative Name – a tanúsítványban felsorolt nevek.
SBOM	Software Bill of Materials – az image összetevőinek jegyzéke.
SNI	Server Name Indication – a kért kiszolgálónév a TLS-kézfogásban.
TLS	Transport Layer Security – titkosított, hitelesített hálózati kapcsolat.